

Universität Stuttgart
Master of Science

Logical Time

Marco Aiello

Logical vs. Physical Time

- Perfect clock synchronization is not feasible
- But do we need it? Often, we just need to know the (partial order) of events
- **Physical time:** *“Time is the continued sequence of existence and events that occurs in an apparently irreversible succession from the past, through the present, and into the future.”* Wikipedia
- **Logical time:** *“Logical time is the discrete sequence of events ordered by causal relations”*

Happens before relation

- Event ϕ_i happens-before ϕ_j , denoted by $\phi_i \rightarrow \phi_j$ if either:
 1. the two events occurred at the same process and ϕ_i precedes ϕ_j in real-time
 2. ϕ_i is the event sending a message m and ϕ_j is the event receiving m
 3. there exists a sequence $\phi_{i+1} \dots \phi_{j-1}$ such that $\phi_i \rightarrow \phi_{i+1} \rightarrow \dots \rightarrow \phi_{j-1} \rightarrow \phi_j$ (transitivity)

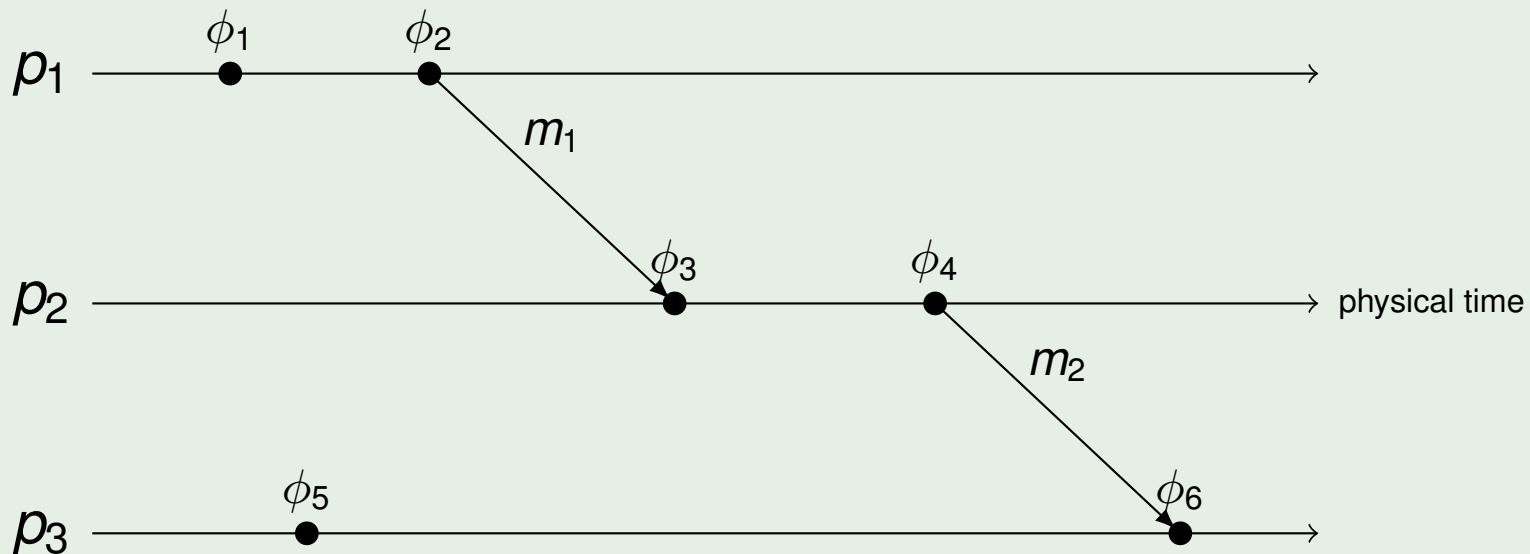
The relation $\rightarrow$ is an irreflexive partial order

If $\phi_i \nrightarrow \phi_j$ and $\phi_j \nrightarrow \phi_i$ then ϕ_i and ϕ_j are concurrent, notation $\phi_i \parallel \phi_j$

Three processes

Example

What is the happened-before relation between the events?



Causal Shuffle

- Given a sequence of events $\sigma = \{\phi_1, \dots, \phi_k\}$, a permutation π of σ is a **causal shuffle** if:
 1. the order of events occurring at individual processors remains unchanged, i.e. $\forall i, 1 \leq i \leq k, \sigma|_i = \pi|_i$, where $|_i$ refers to the events occurring in π .
 2. if a message m is sent during p_i 's event ϕ in σ , then in π , ϕ precedes the delivery of m .

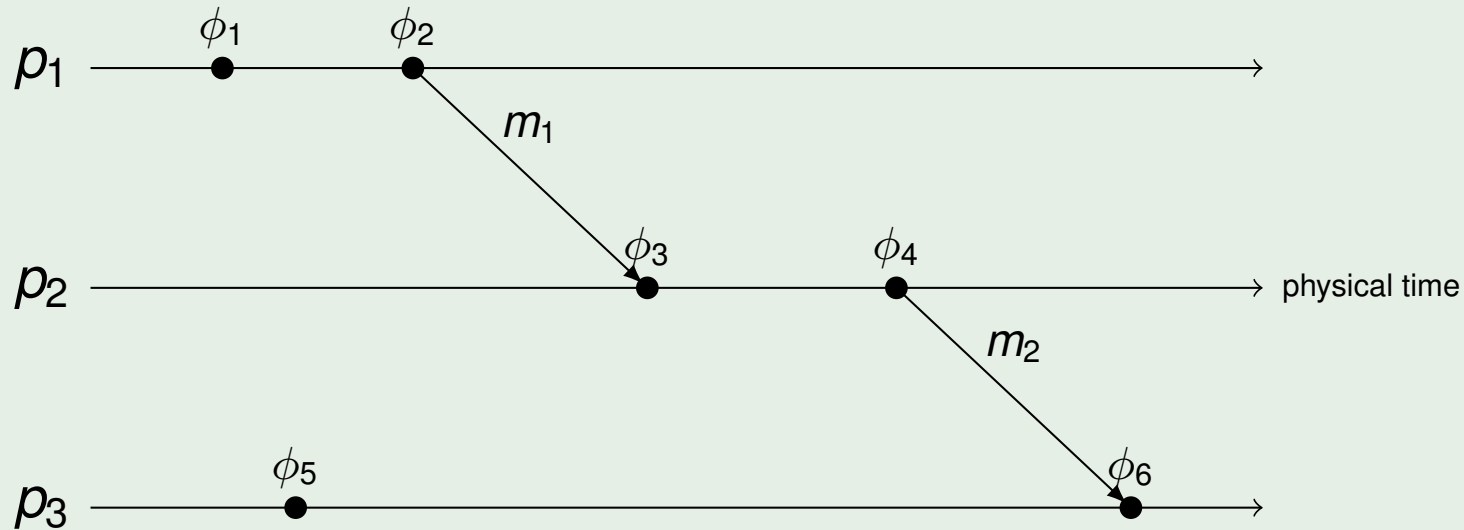
The resulting sequence π is indistinguishable to the processors.

Lemma: Any total ordering of the events in σ that is consistent with the $\rightarrow$ relation is a causal shuffle of σ .

Three processes

Example

What is the happened-before relation between the events?



- Causal shuffle?
 - $\phi_1, \phi_3, \phi_4, \phi_2$
 - $\phi_3, \phi_1, \phi_2, \phi_4$

Lamport Timestamps

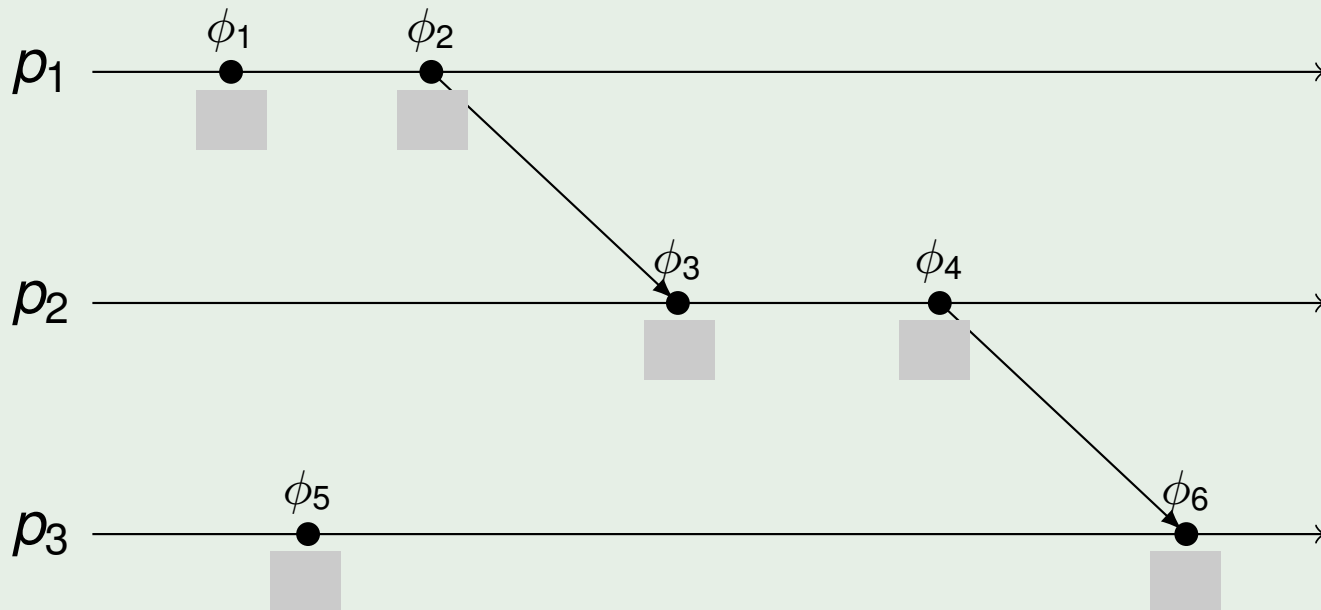
The Lamport Timestamp $LT(\phi)$ is a counter providing a value to each event in the system:

- Each p_i keeps a local counter LT_i which is initially set to 0
- At each event ϕ in p_i ,
$$LT_i = \max\{LT_i, \max\{LT < \text{msgs received upon } \phi > \}\} + 1$$
- When p_i sends a message, it attaches the current LT_i value to the message

For each p_i , LT_i is strictly increasing

Lamport Timestamps

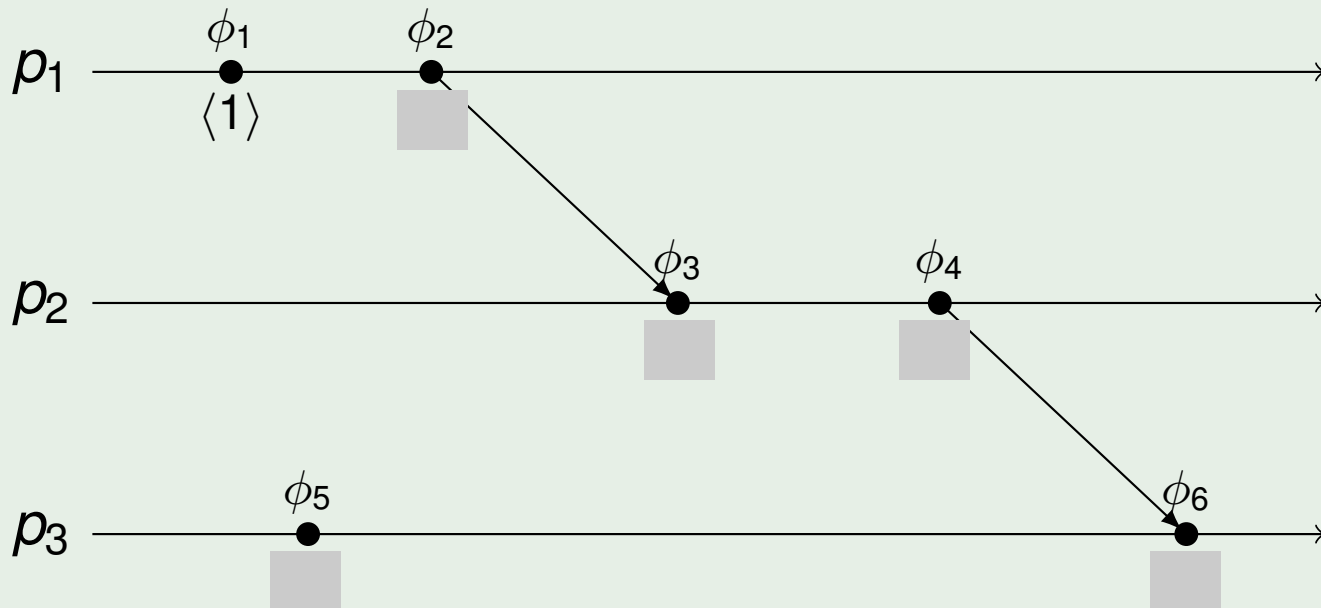
Example



Note that $LT(\phi_5) < LT(\phi_4)$ but $\neg(\phi_5 \rightarrow \phi_4)$

Lamport Timestamps

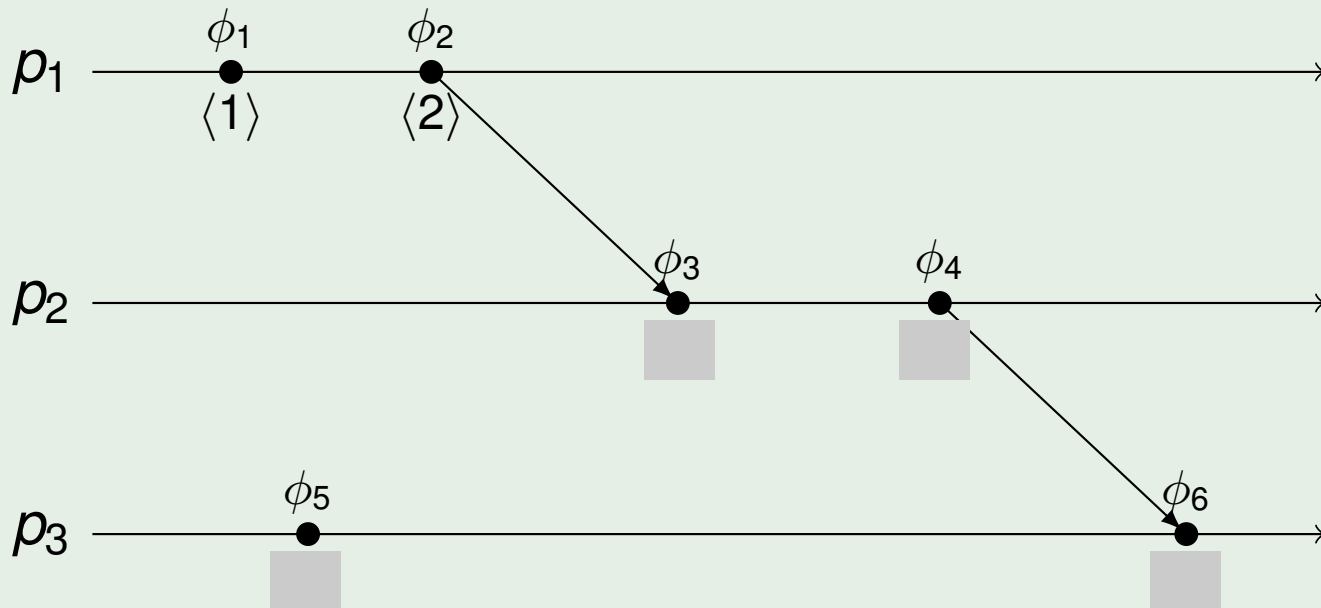
Example



Note that $LT(\phi_5) < LT(\phi_4)$ but $\neg(\phi_5 \rightarrow \phi_4)$

Lamport Timestamps

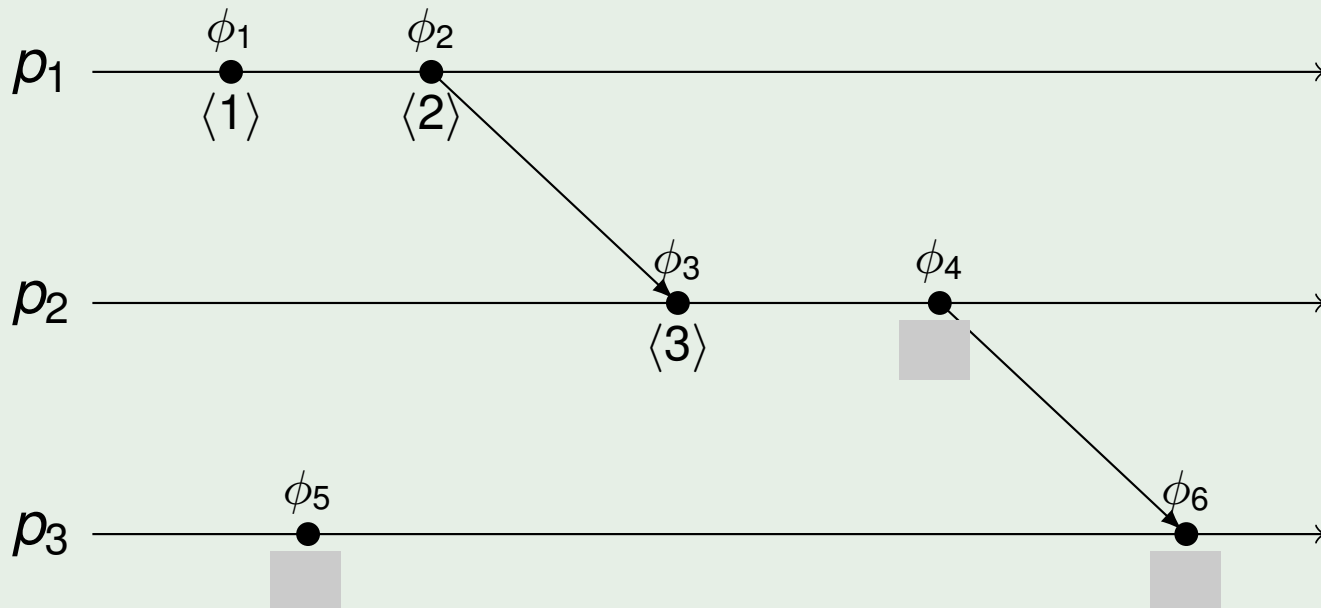
Example



Note that $LT(\phi_5) < LT(\phi_4)$ but $\neg(\phi_5 \rightarrow \phi_4)$

Lamport Timestamps

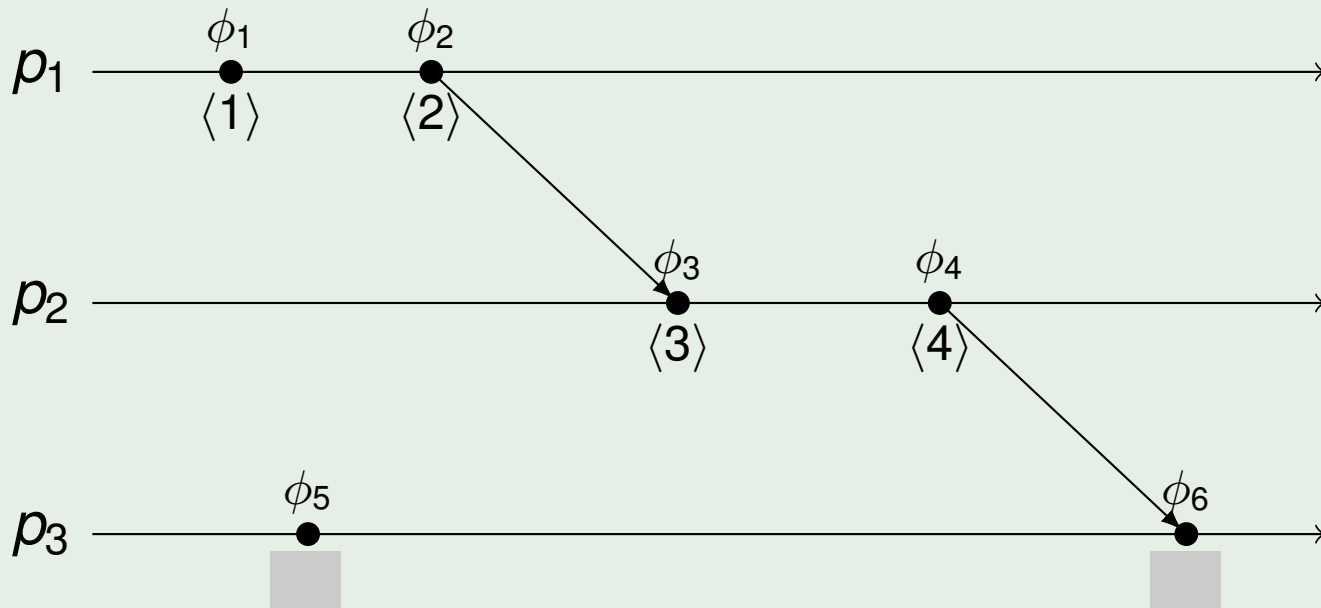
Example



Note that $LT(\phi_5) < LT(\phi_4)$ but $\neg(\phi_5 \rightarrow \phi_4)$

Lamport Timestamps

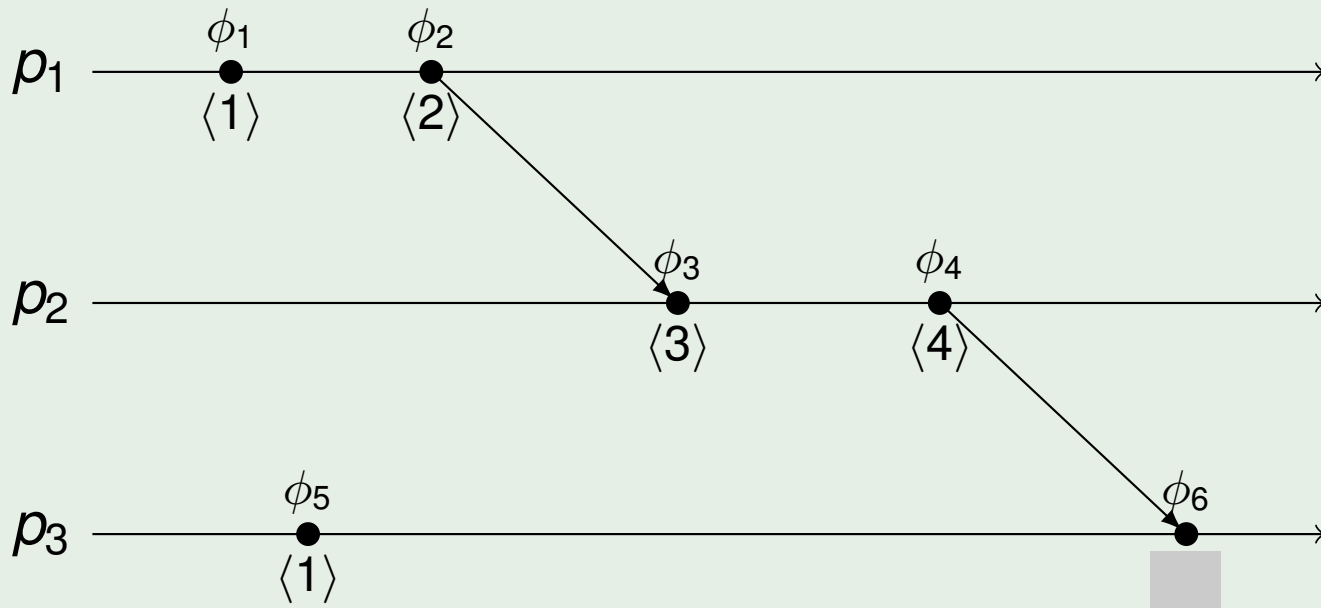
Example



Note that $LT(\phi_5) < LT(\phi_4)$ but $\neg(\phi_5 \rightarrow \phi_4)$

Lamport Timestamps

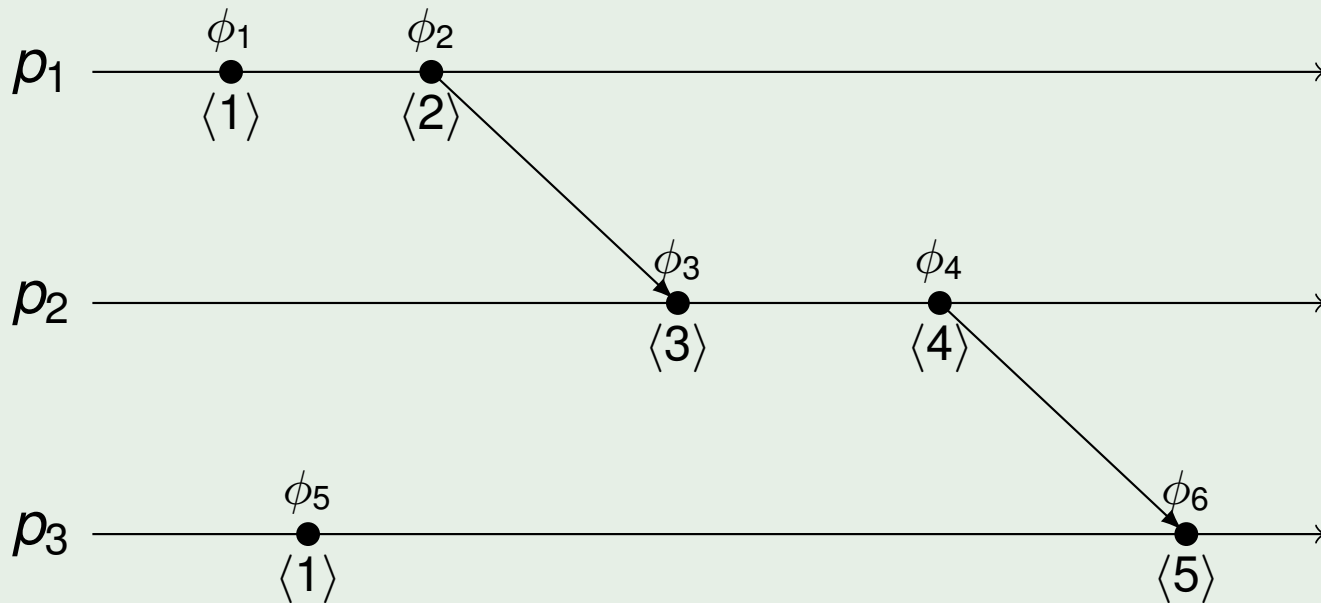
Example



Note that $LT(\phi_5) < LT(\phi_4)$ but $\neg(\phi_5 \rightarrow \phi_4)$

Lamport Timestamps

Example



Note that $LT(\phi_5) < LT(\phi_4)$ but $\neg(\phi_5 \rightarrow \phi_4)$

Weak Consistency

Theorem: Let ϕ_1, ϕ_2 be two events in an execution. If $\phi_1 \rightarrow \phi_2$ then $LT(\phi_1) < LT(\phi_2)$

If $LT(\phi_1) < LT(\phi_2)$, we can only tell that $\neg(\phi_2 \rightarrow \phi_1)$, but we don't know whether $\phi_1 \rightarrow \phi_2$ or $\phi_1 \parallel \phi_2$

The problem is that $<$ induces a *total order* over integers while $\rightarrow$ a *partial order*, so the non-causality relation is lost.

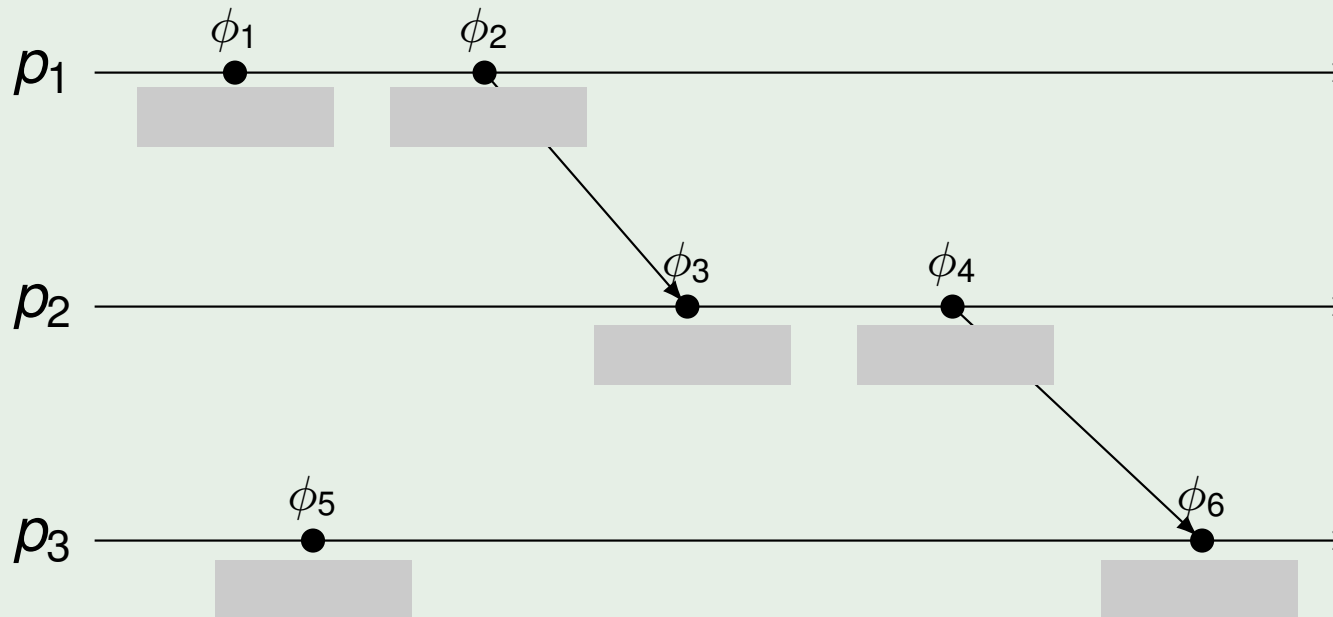
Vector clocks

The Vector timestamp is computed as follows:

- Each p_i keeps a local vector of size n VC_i , whose entries $VC_i[j]$ are initially set to 0
- At each event ϕ in p_i , $VC_i[i] = VC_i[i] + 1$ and for all $j \neq i$
 $VC_i[j] = \max\{VC_i[j], \max\{VC[j] < \text{msgs received upon } \phi > \}\}$
- When p_i sends a message, it attaches the current VC_i value to the message

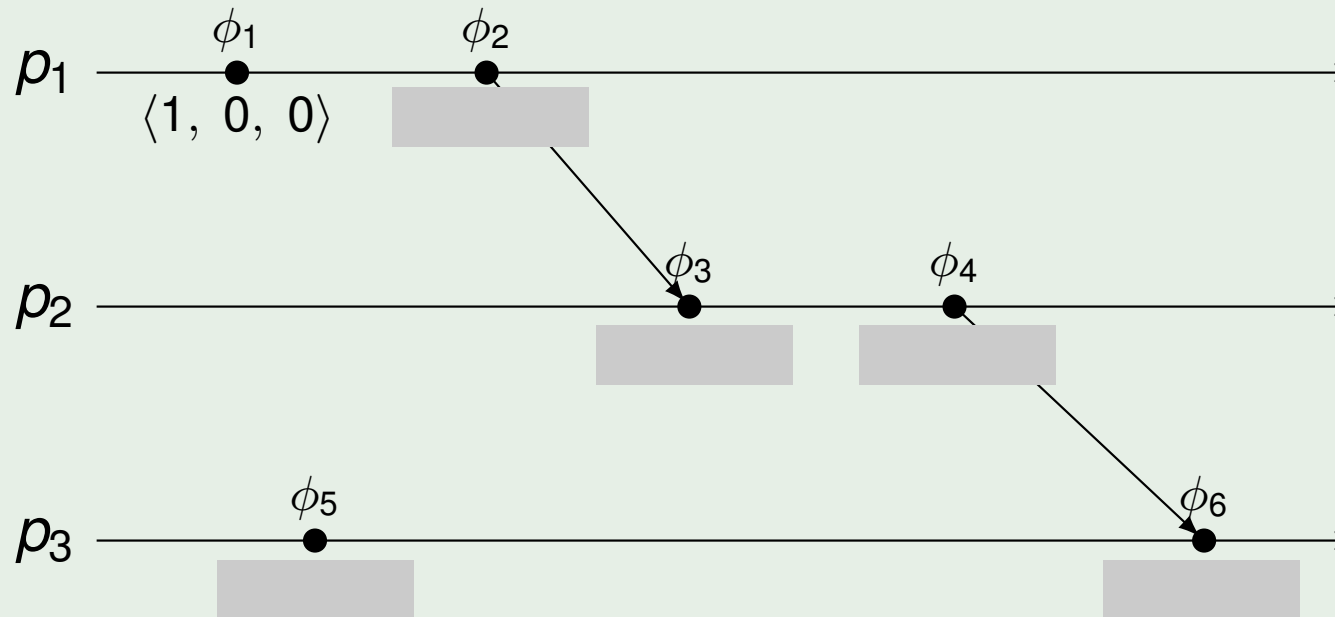
Computing the vector clock

Example



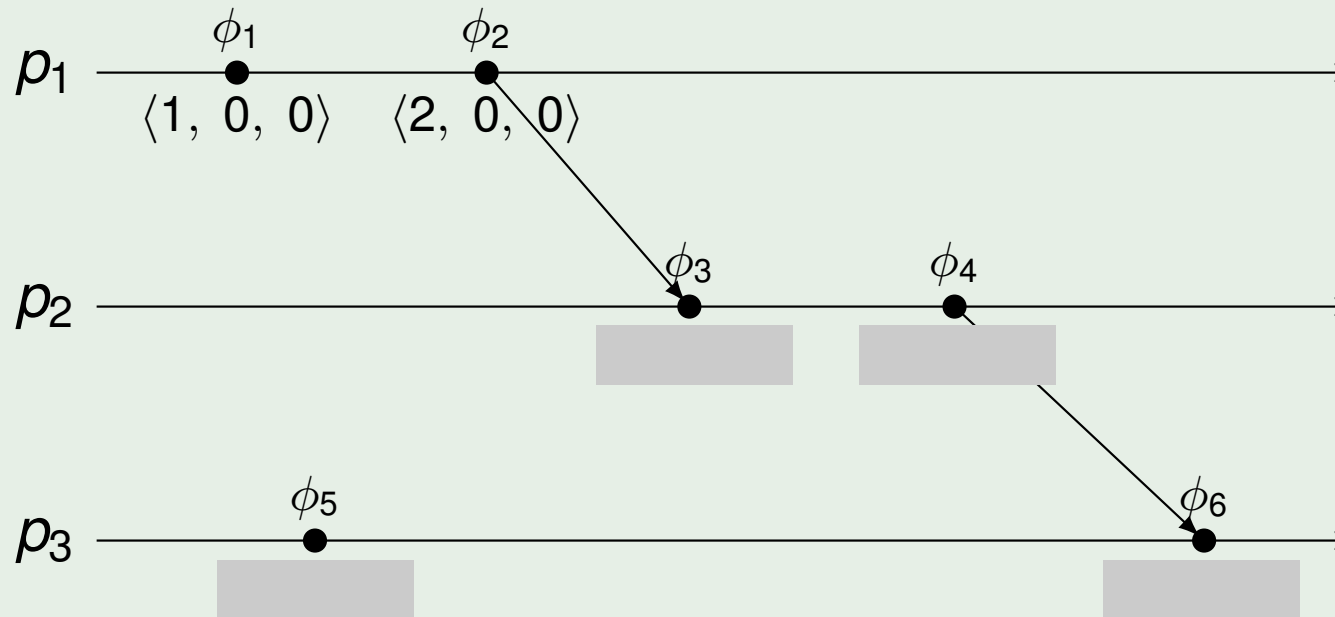
Computing the vector clock

Example



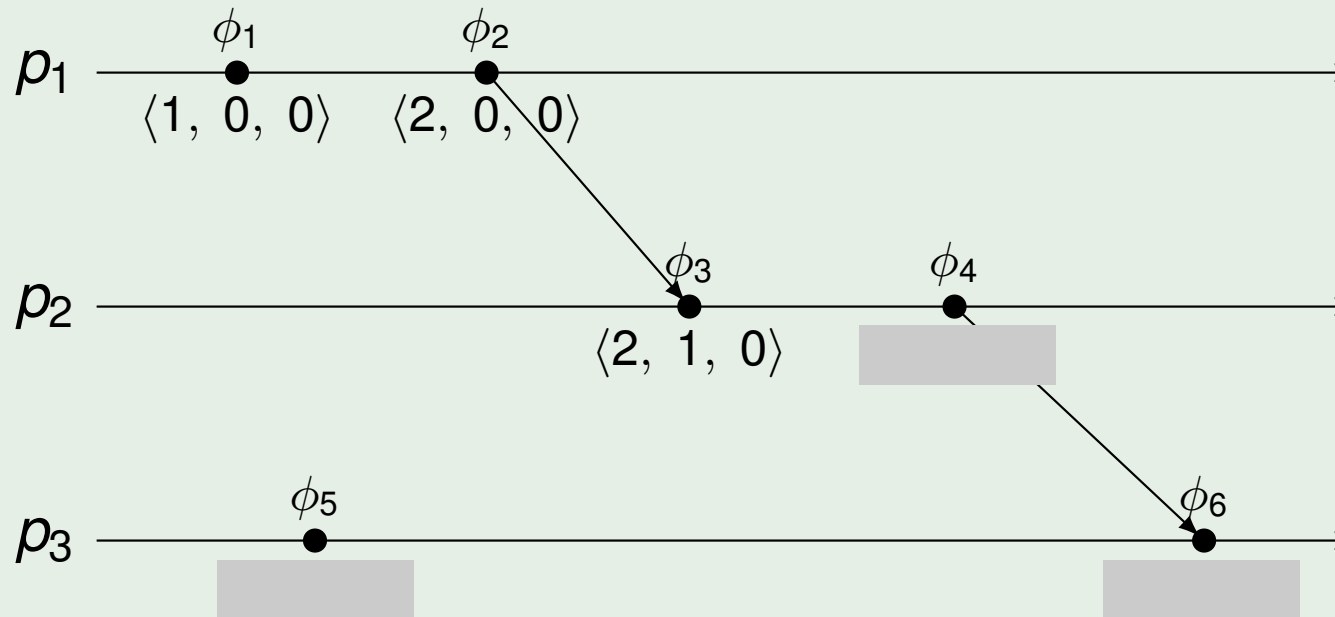
Computing the vector clock

Example



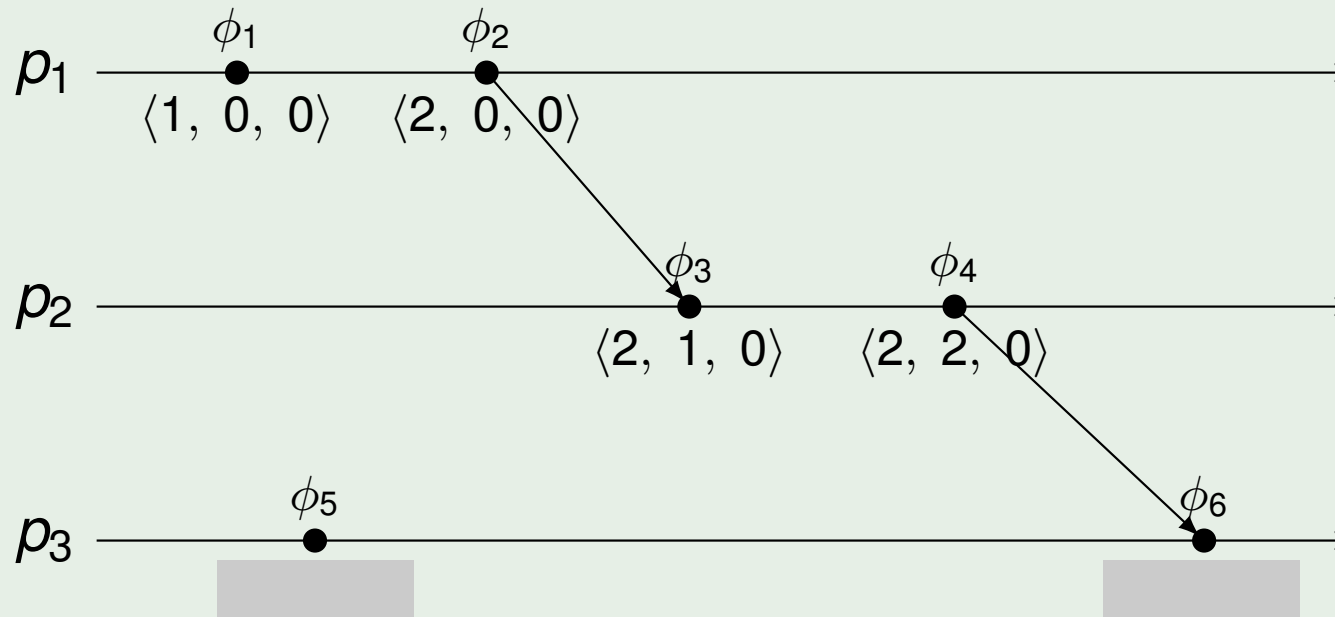
Computing the vector clock

Example



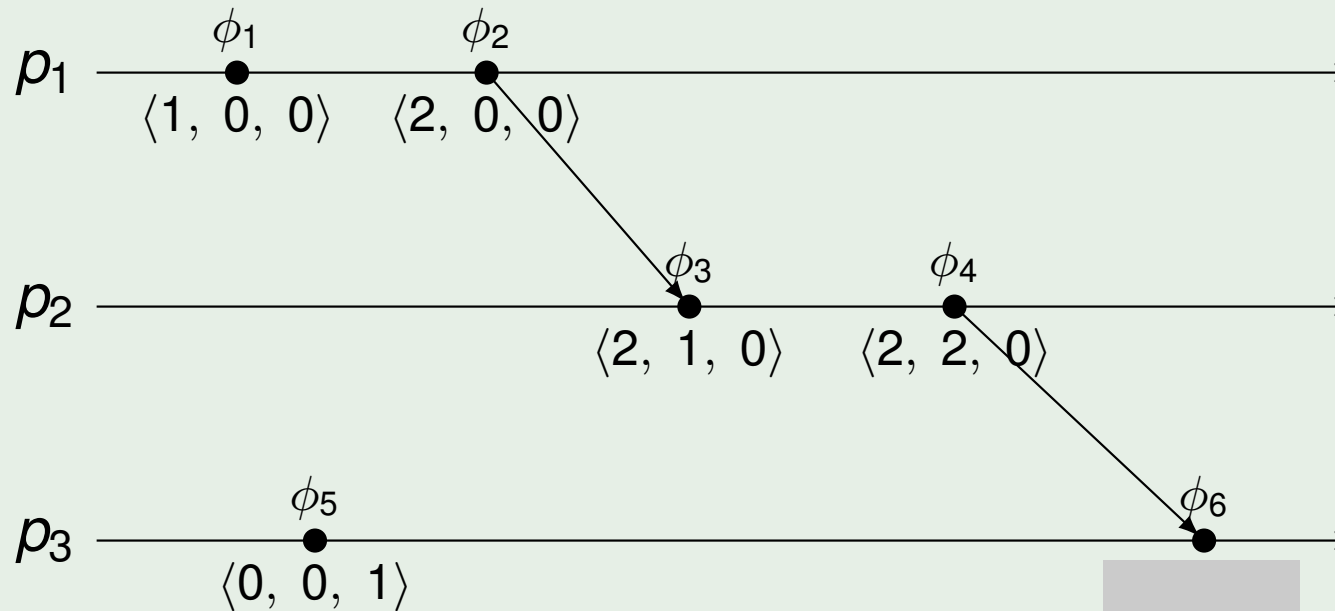
Computing the vector clock

Example



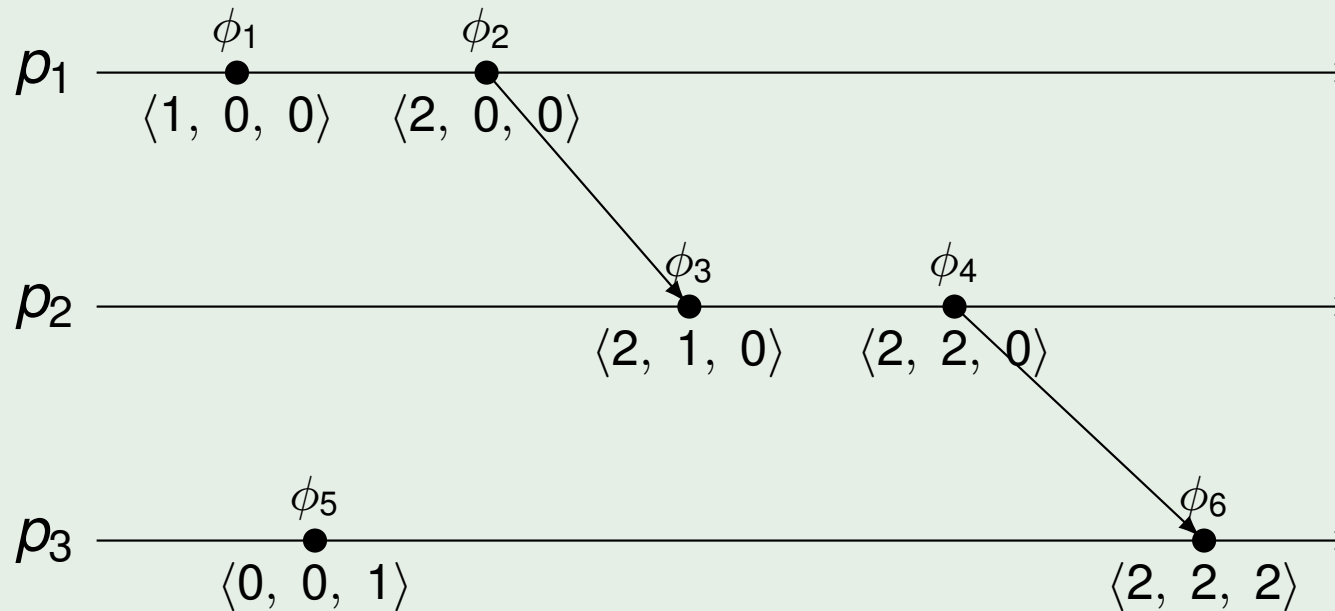
Computing the vector clock

Example



Computing the vector clock

Example



Vector clocks and concurrency

Proposition: For every p_i , $VC_i[j] \leq VC_j[j]$ for all i, j $1 \leq i, j \leq n$

Vector clocks capture concurrency, i.e., it holds that $\phi_i \parallel \phi_j$ iff $VC(\phi_i)$ and $VC(\phi_j)$ are incomparable

Recall that:

$VC_i \leq VC_j$ iff for all $1 \leq k \leq n$ $VC_i[k] \leq VC_j[k]$

$VC_i < VC_j$ iff $VC_i \leq VC_j \wedge VC_i \neq VC_j$

$VC_i \parallel VC_j$ iff for all $\neg(VC_i \leq VC_j) \wedge \neg(VC_j \leq VC_i)$

Theorem (Strong Consistency): $\phi_i \rightarrow \phi_j$ iff $VC(\phi_i) < VC(\phi_j)$

Proof of strong consistency

$\Rightarrow$ If ϕ_i, ϕ_j at the same p_i , trivial. If ϕ_i at p_i sends message received by ϕ_j at p_j , then $VC_j(\phi_j)[k] > VC_i(\phi_i)[k]$ for all $k \neq j$ and $VC_j(\phi_j)[j] = VC_i(\phi_i)[j] + 1$.
Rest by transitivity of $<$ relation of vectors

$\Leftarrow$ If $\phi_j \rightarrow \phi_i$, contradiction. If $\phi_i \parallel \phi_j$ then $VC_j(\phi_j)[i] < VC_i(\phi_i)[i]$ since the only way that $VC_j(\phi_j)[i] = VC_i(\phi_i)[i]$ would be the existence of a sequence of events ϕ'_i such that $\phi_i \rightarrow \phi'_1 \rightarrow \dots \rightarrow \phi'_n \rightarrow \phi_j$. Similarly $VC_i[j] < VC_j[j]$.
Thus, $VC_i(\phi_i), VC_j(\phi_j)$ are incomparable.

Lower bound

Theorem (Lower bound on the size of vector clocks): If VC is a function that maps each event in an execution in a system of n processors to a vector in S^k , where S is any totally ordered set (e.g. $\mathbb{R}$), in a manner that captures concurrency, then $k \geq n$.

Recording a meaningful global state

- No omniscient observer to record the system's global state, i.e. the set of the processors' local states, as well as the state of each channel in which messages flow.
- **Snapshot problem:** compute a meaningful global state so that it looks to the processors as if the snapshot was taken at the same instance everywhere in the system.
- Processors have to compute an approximate snapshot of the global state that captures the notion of causality (every message that is recorded as received is recorded as sent).

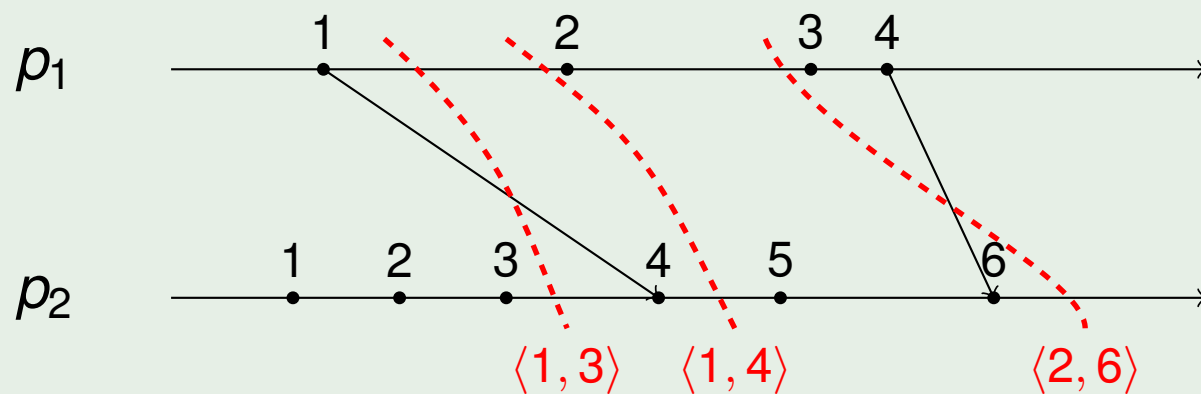
Cut

- A **cut** of an execution is an n -vector $\vec{k} = \langle k_1, \dots, k_n \rangle$ of positive integers, where k_i indicates the number of events taken by p_i .
- A **global state** $S^k = (s_1, \dots, s_n, c_1, \dots, c_m)$ of the cut $\vec{k}$ is the set of states of the processors and the channels connecting them
- A cut $\vec{k}$ is **consistent** if for all $1 \leq i, j \leq n$ the $(k_i + 1)^{\text{st}}$ event on p_i does not happen before the k_j^{th} event on p_j . I.e. for each event included in a consistent cut, all events that happened before this event must also be included in it. A global state corresponding to a consistent cut is consistent.

Which cut is consistent?

Example

Some consistent and inconsistent cuts



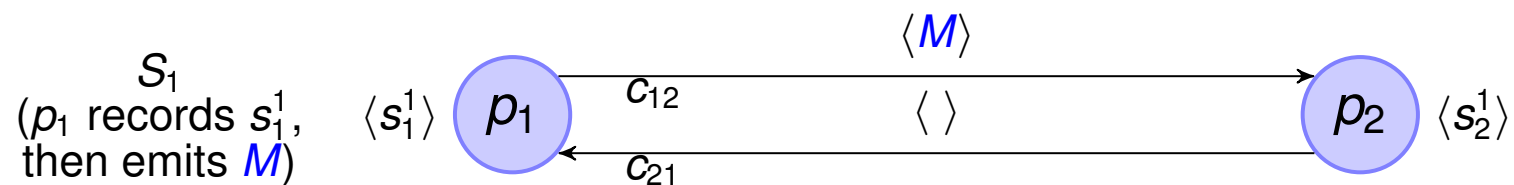
Chandy-Lamport snapshot algorithm

- It is a distributed algorithm
- Initiated by any node
- Assumptions:
 1. No failures. All messages arrive intact and only once.
 2. Communication channels are unidirectional and deliver messages in FIFO order (guarantees that the computed global state is consistent).
 3. There is a path between any two processors, i.e., the graph of processors and communication channels is strongly connected (guarantees termination).
 4. The snapshot algorithm does not interfere with the normal execution of the processes.

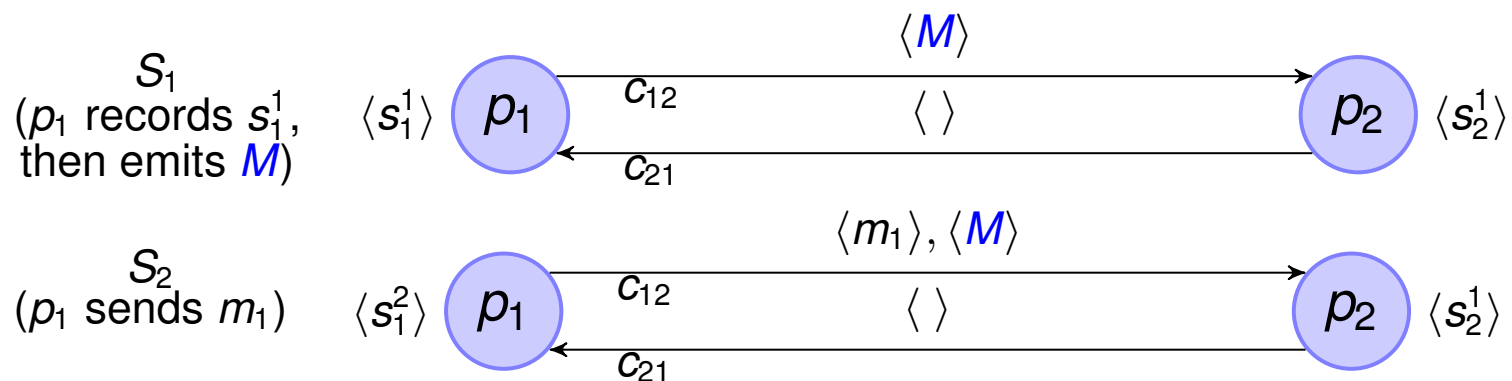
Snapshot Algorithm

- Each p_i that wants to initiate a snapshot records its local state s_i , sends a special marker message $\langle M \rangle$ to all outgoing channels and starts recording messages arriving over its incoming channels.
- When a p_i receives a $\langle M \rangle$ over channel c and has not yet recorded its state, it:
 - a. records its local state and the state of c as empty
 - b. sends a $\langle M \rangle$ to all outgoing channels
 - c. starts recording messages arriving over the other incoming channels
- When a p_i receives an $\langle M \rangle$ over c and has already saved its state, it records the state of c as the set of messages recorded over c (channel states account for msgs that arrived after the receiver recorded its state and were sent before the sender recorded its own state)

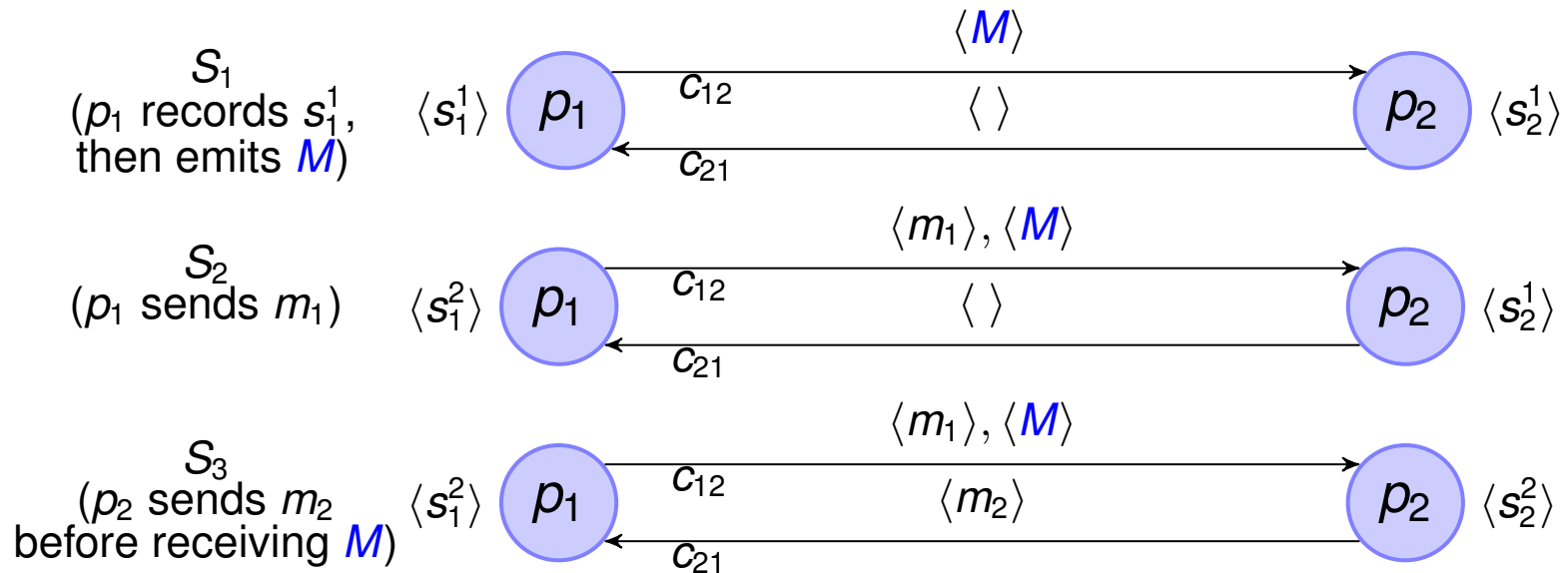
Simple Example



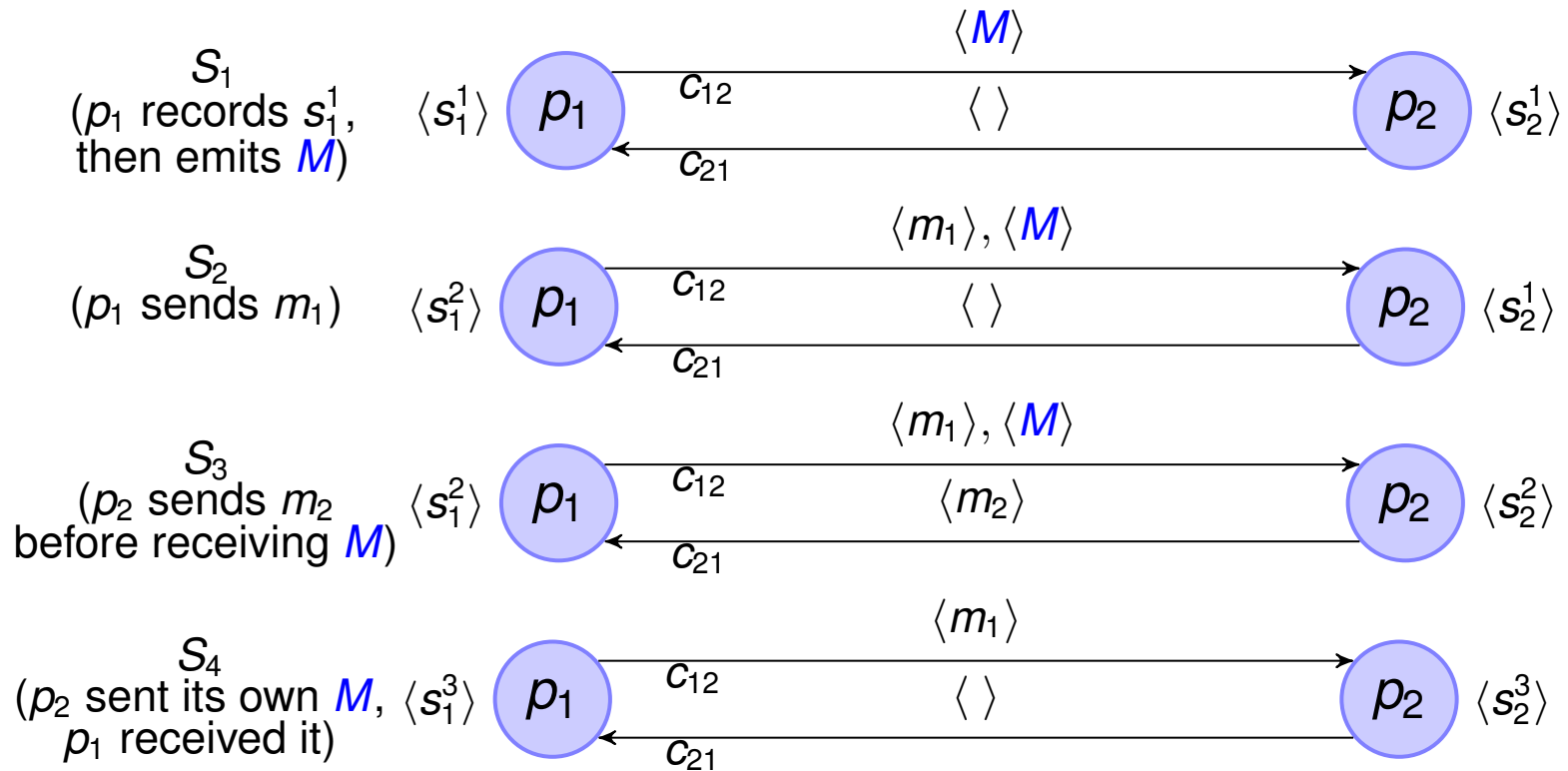
Simple Example



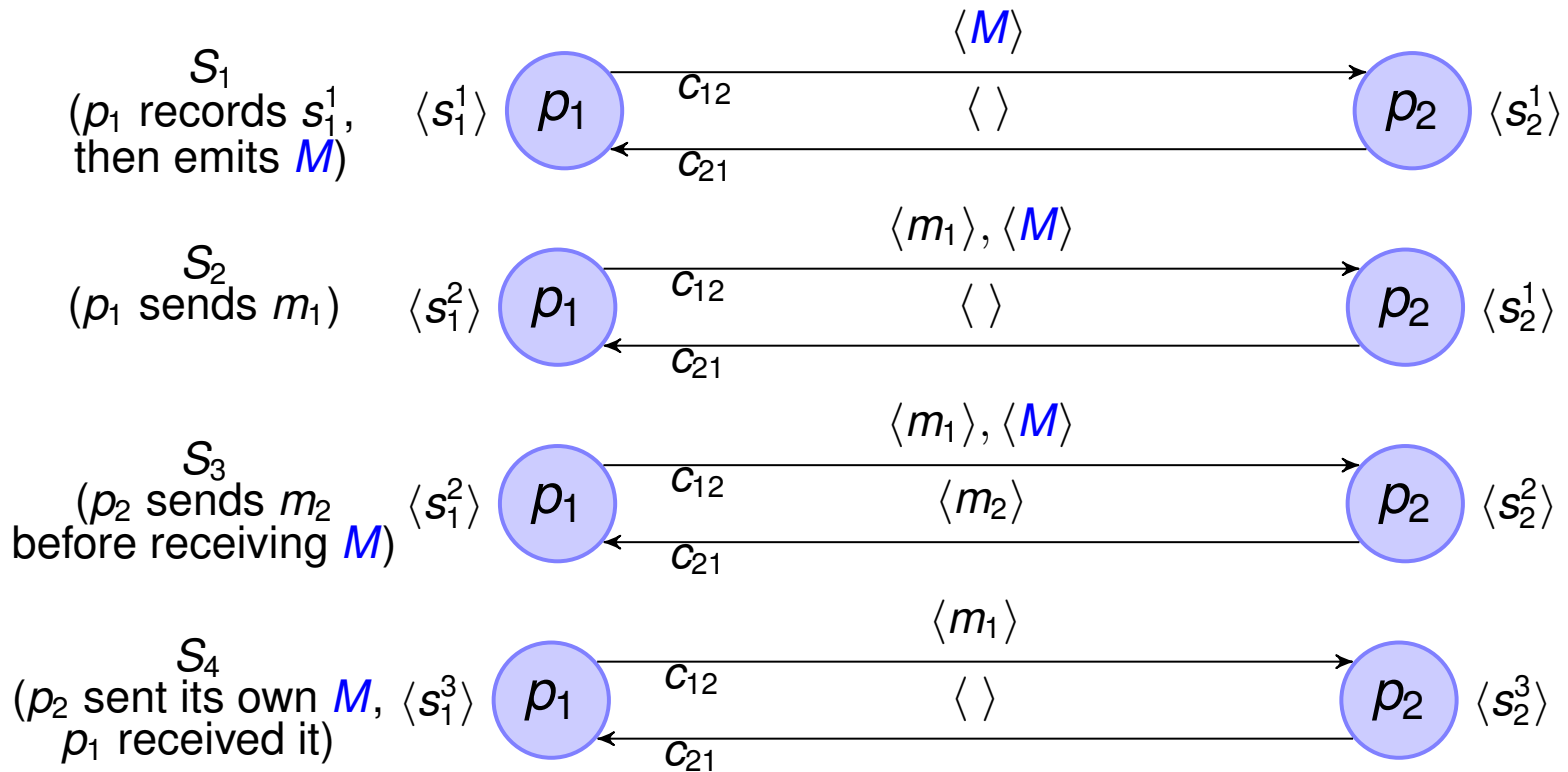
Simple Example



Simple Example



Simple Example



$$S^* : (s_1 = s_1^1, s_2 = s_2^2, c_{12} = \langle \rangle, c_{21} = \langle m_2 \rangle) \notin \{S_1, S_2, S_3, S_4\}$$

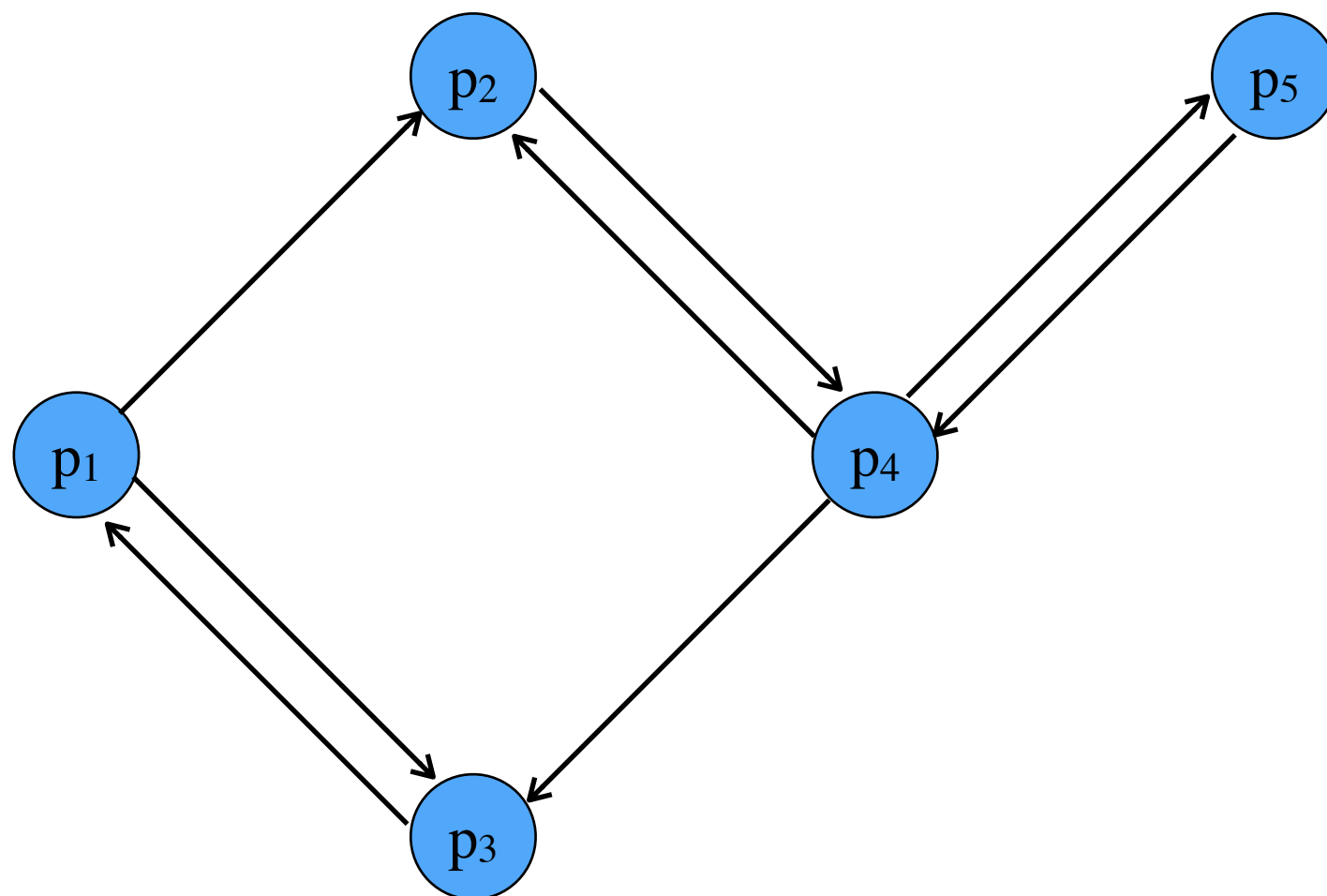
Reachability of the recorded state

Theorem: Let S_i be the global state immediately before the first process recorded its state, and S_f the global state immediately after the last state-recording action. Let seq be the sequence of events that takes the system from S_i to S_f . Then, there exists a sequence seq' that is a causal shuffle of seq such that the recorded global state S^* is reachable from S_i and S_f is reachable from S_i .

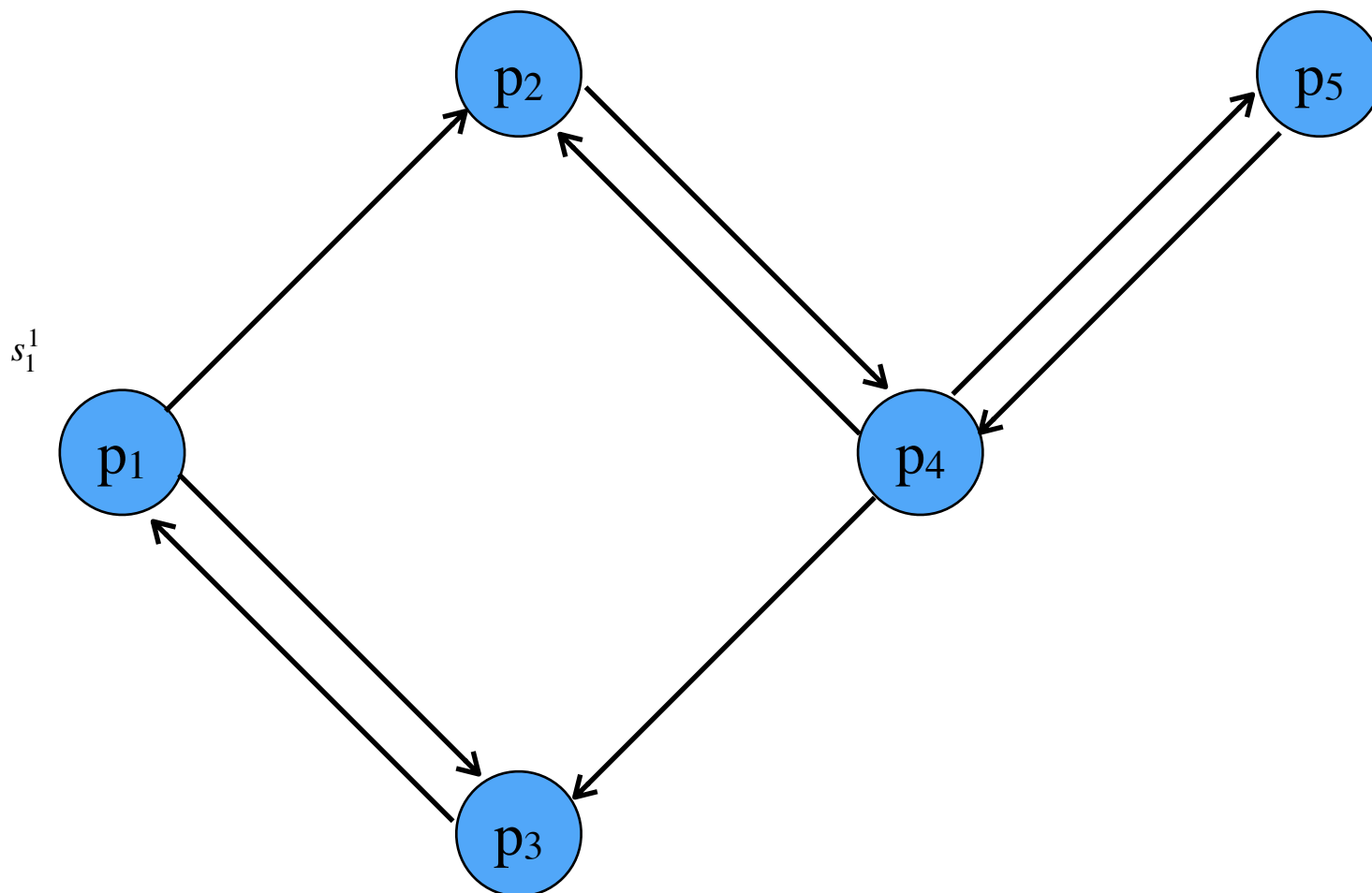
Facts

- The delivered global state S^* may differ from all actual global states through which the system passed.
- However, the system could have passed through S^* in some equivalent executions.
- If a stable property p is true in S^* , we can conclude that it is true in S_f (the converse doesn't hold).
- If a stable property is false in S^* then we can conclude it is false in S_i (the converse doesn't hold).
- Examples of stable properties: deadlock, termination, loss of a token.

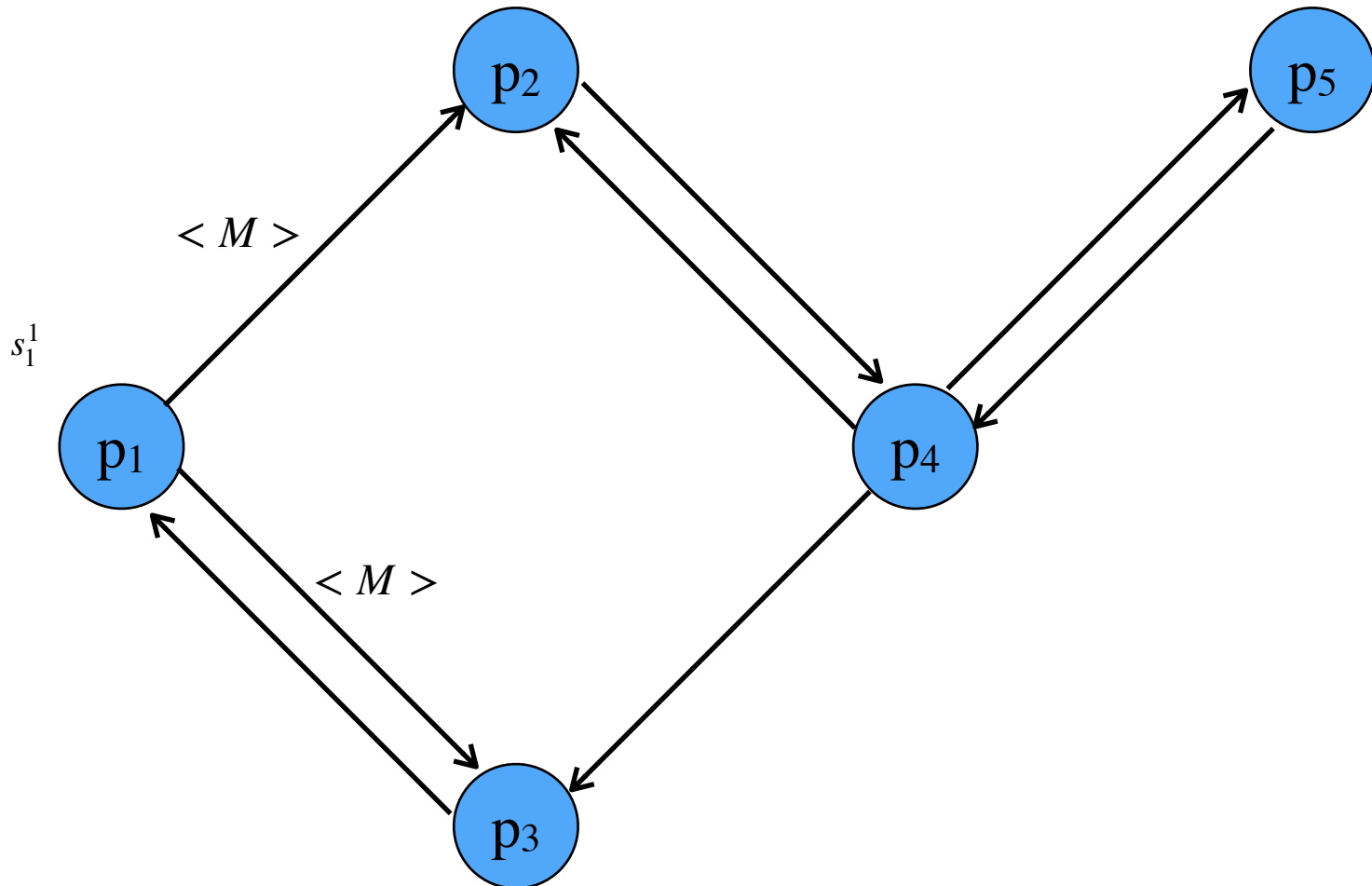
Larger example



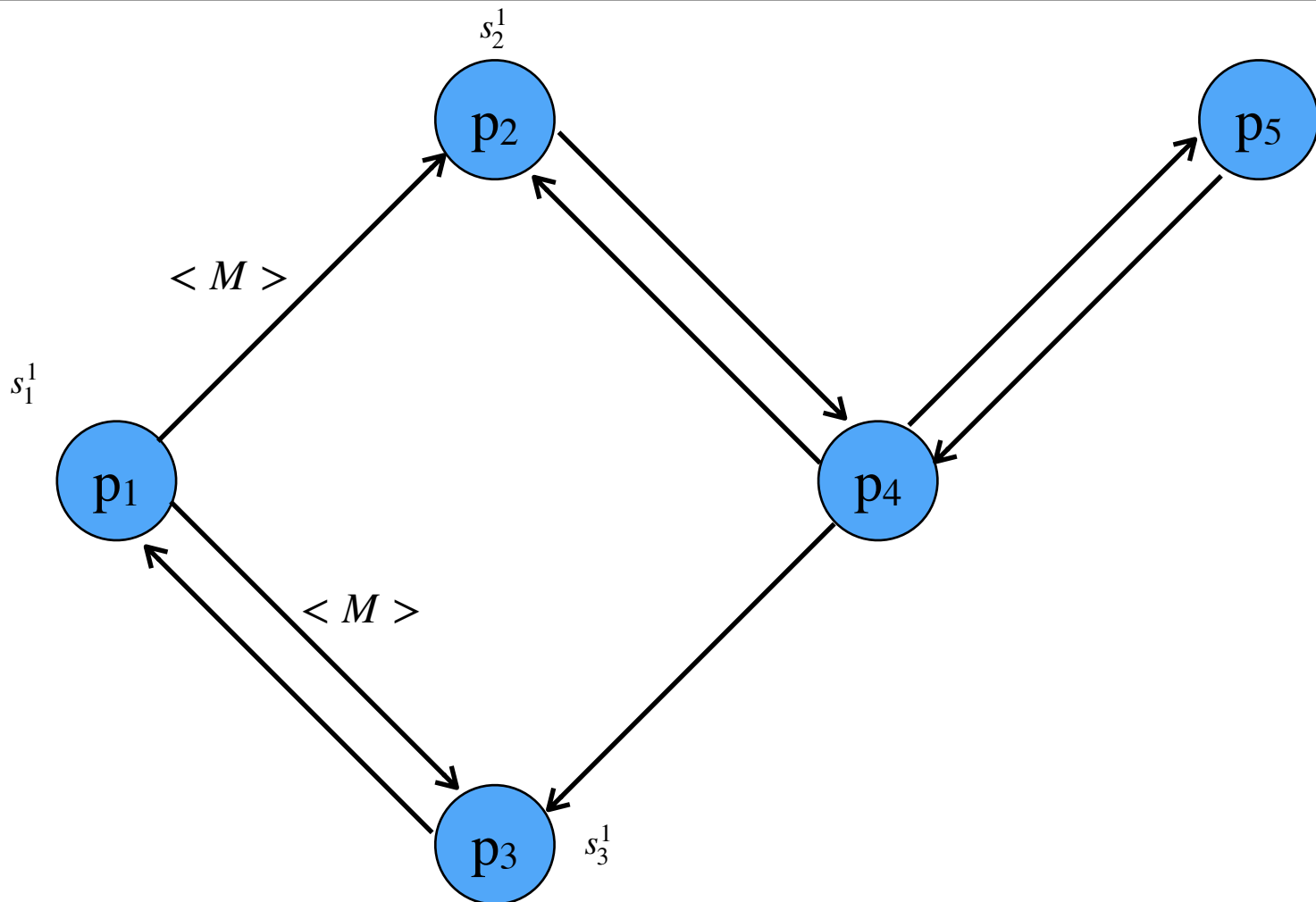
Larger example



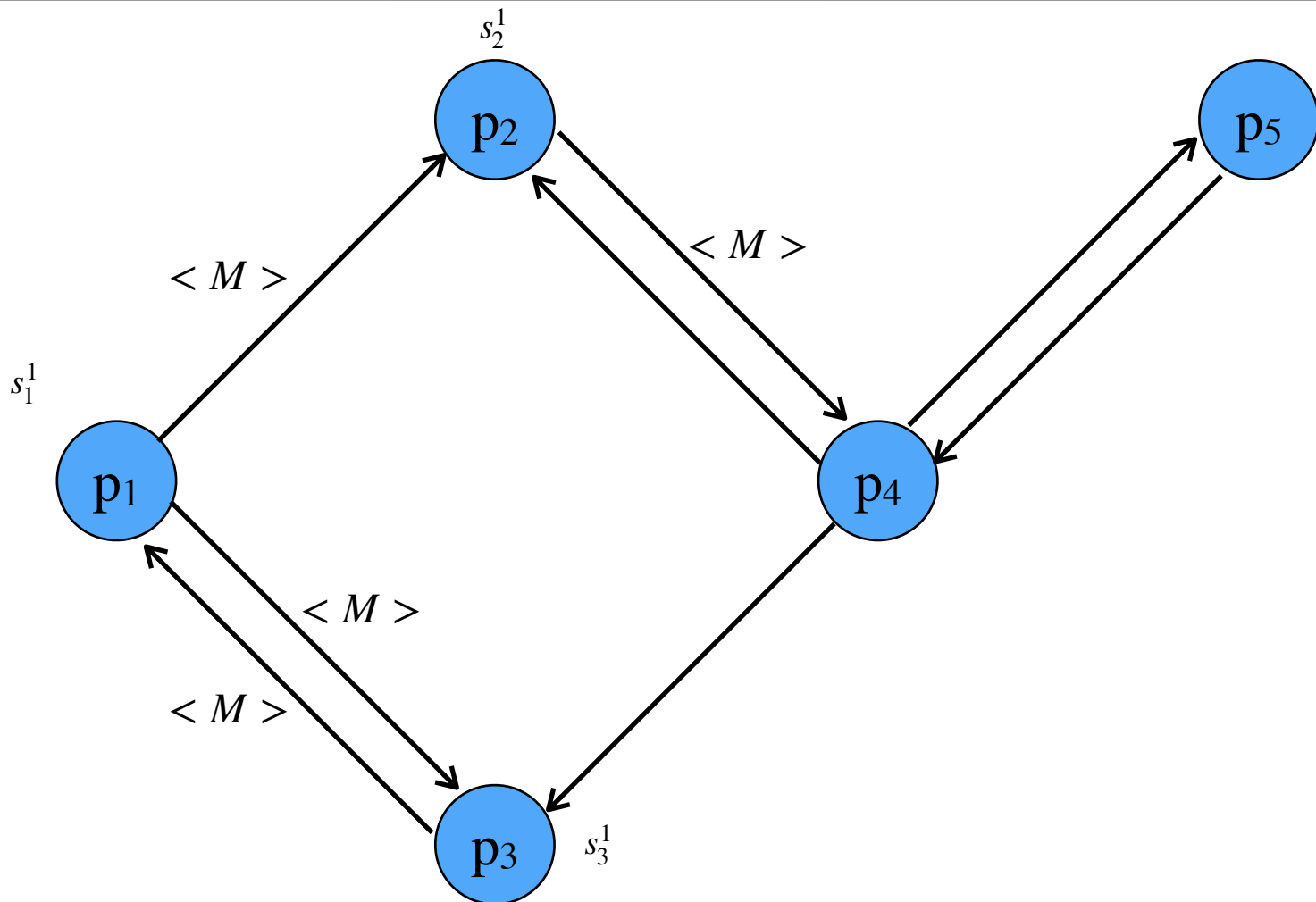
Larger example



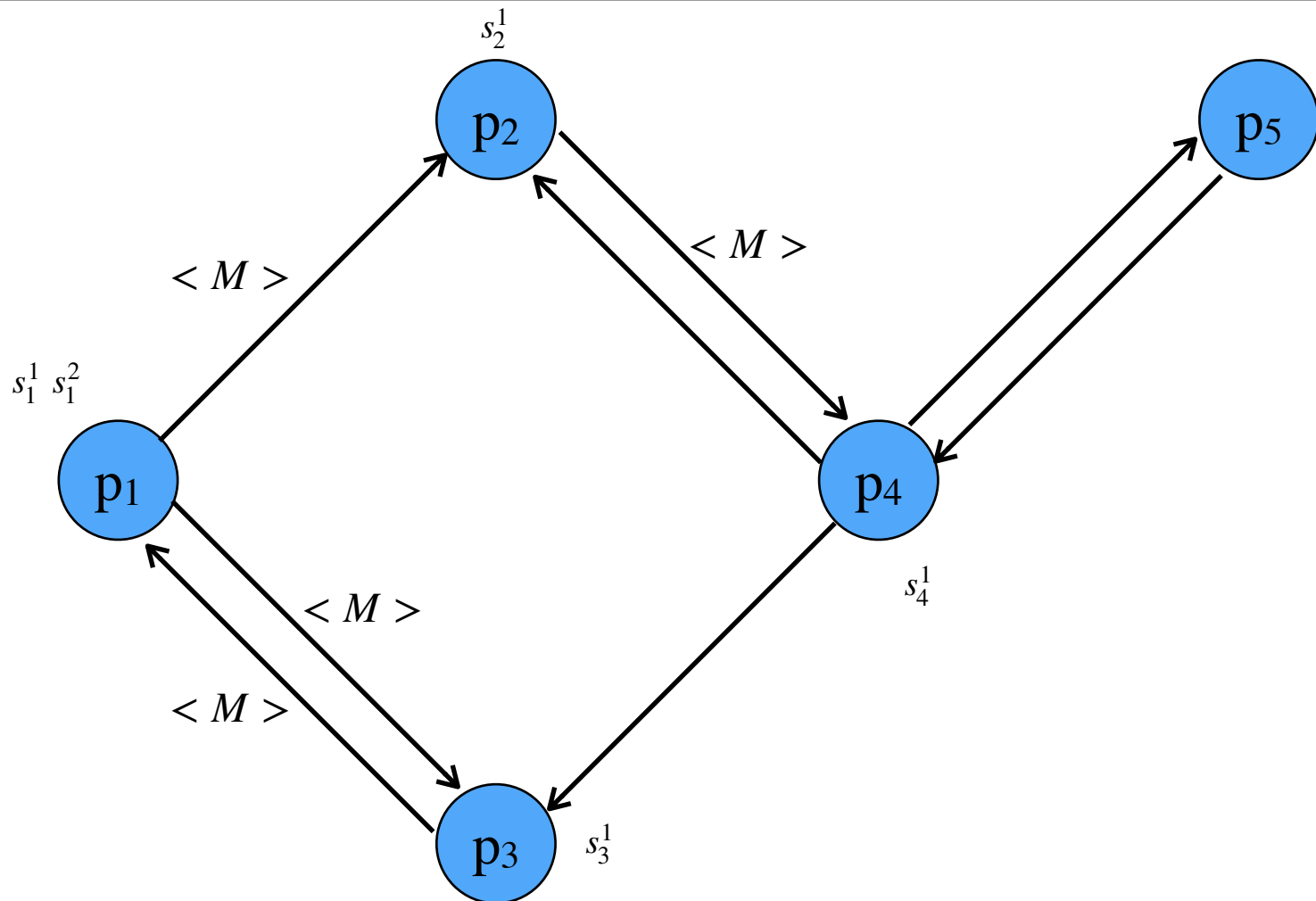
Larger example



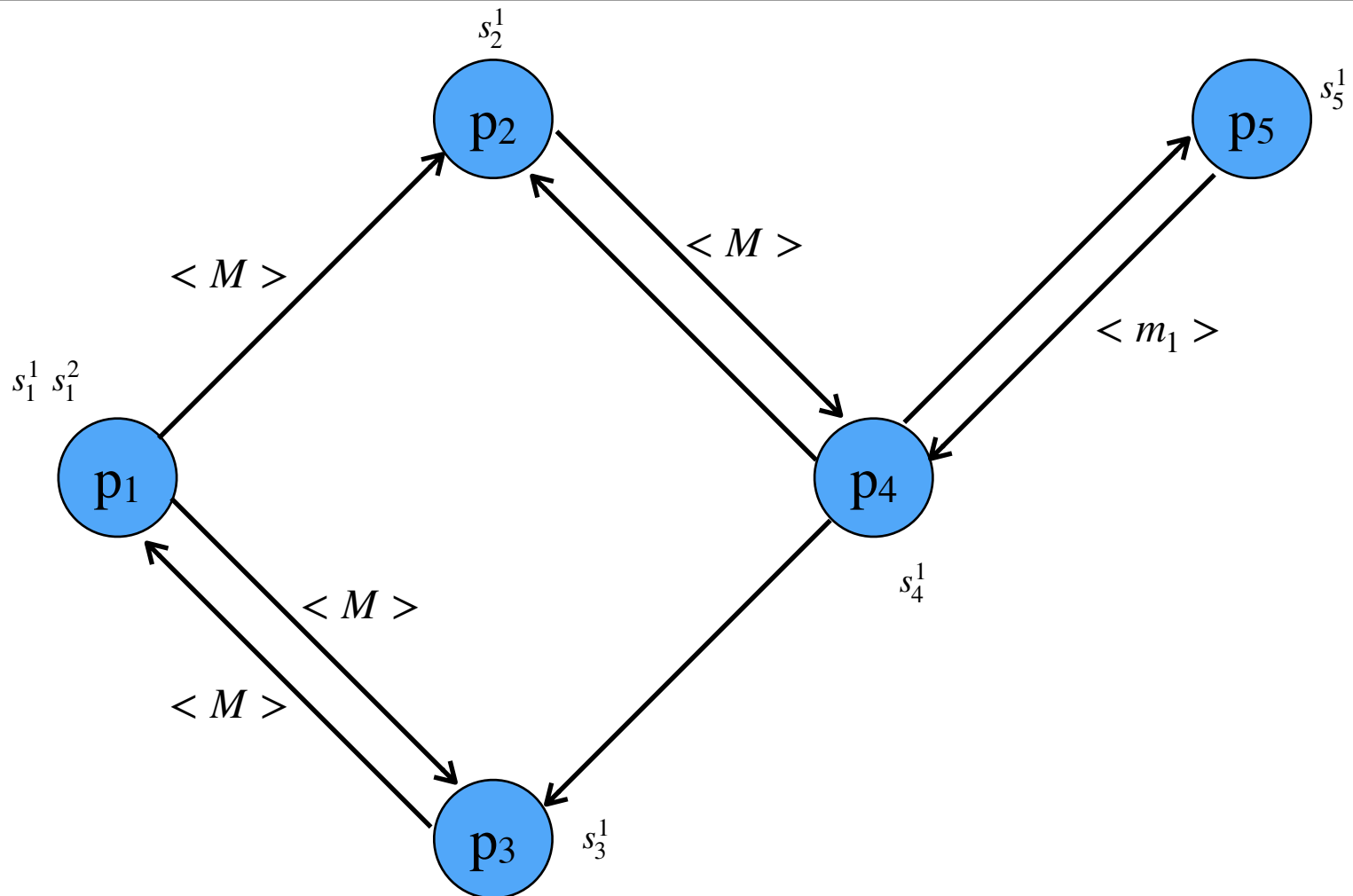
Larger example



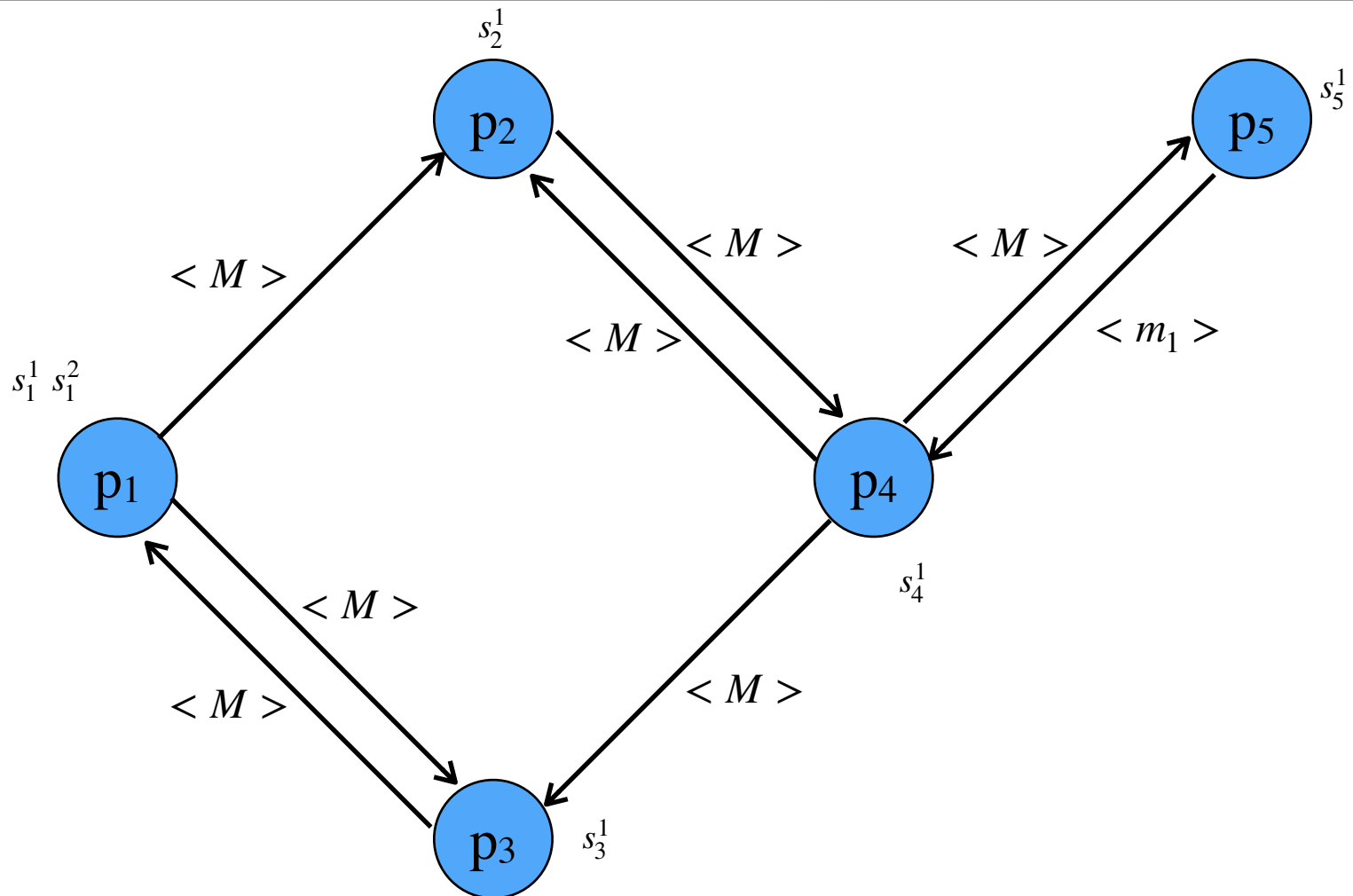
Larger example



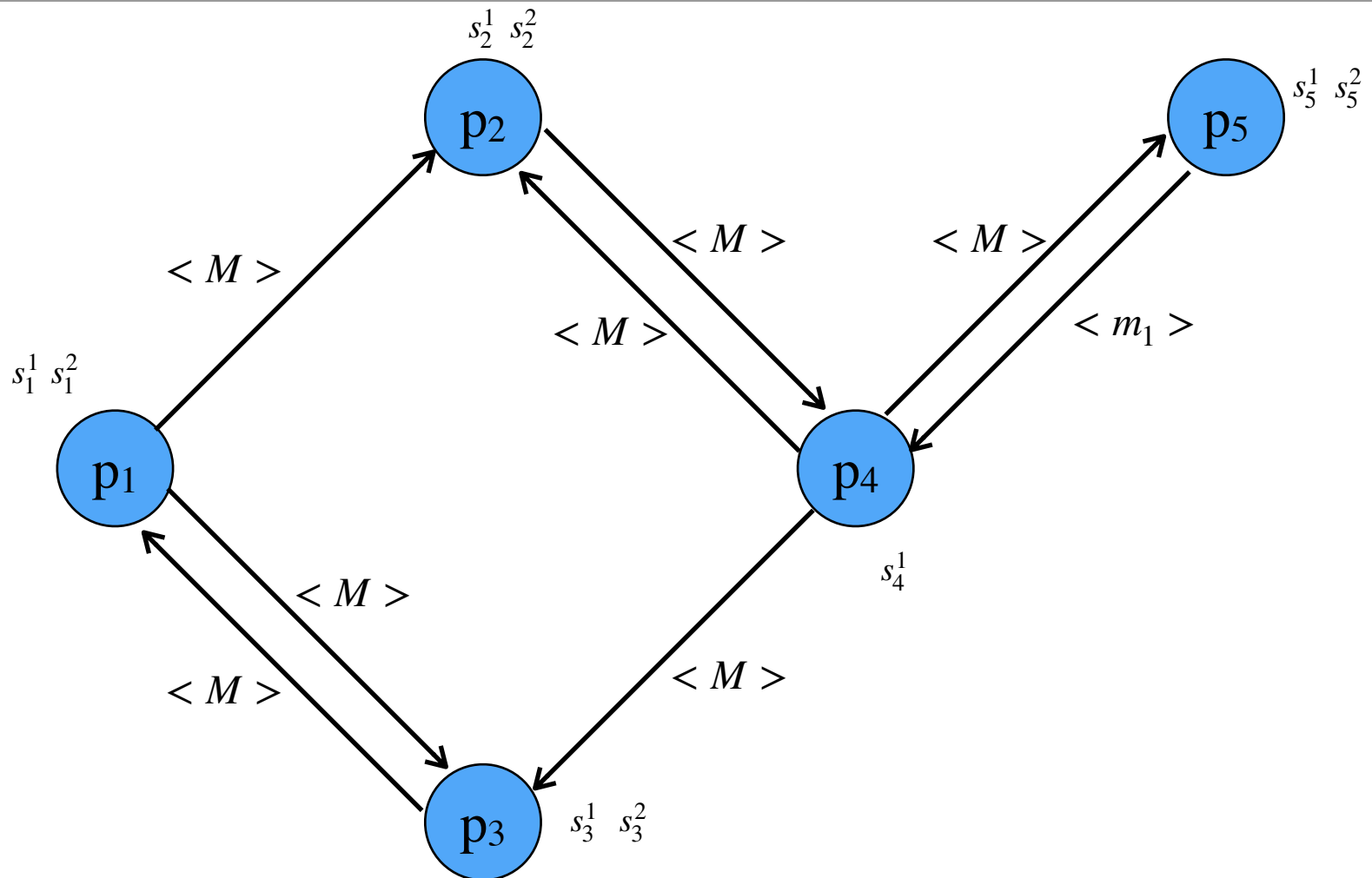
Larger example



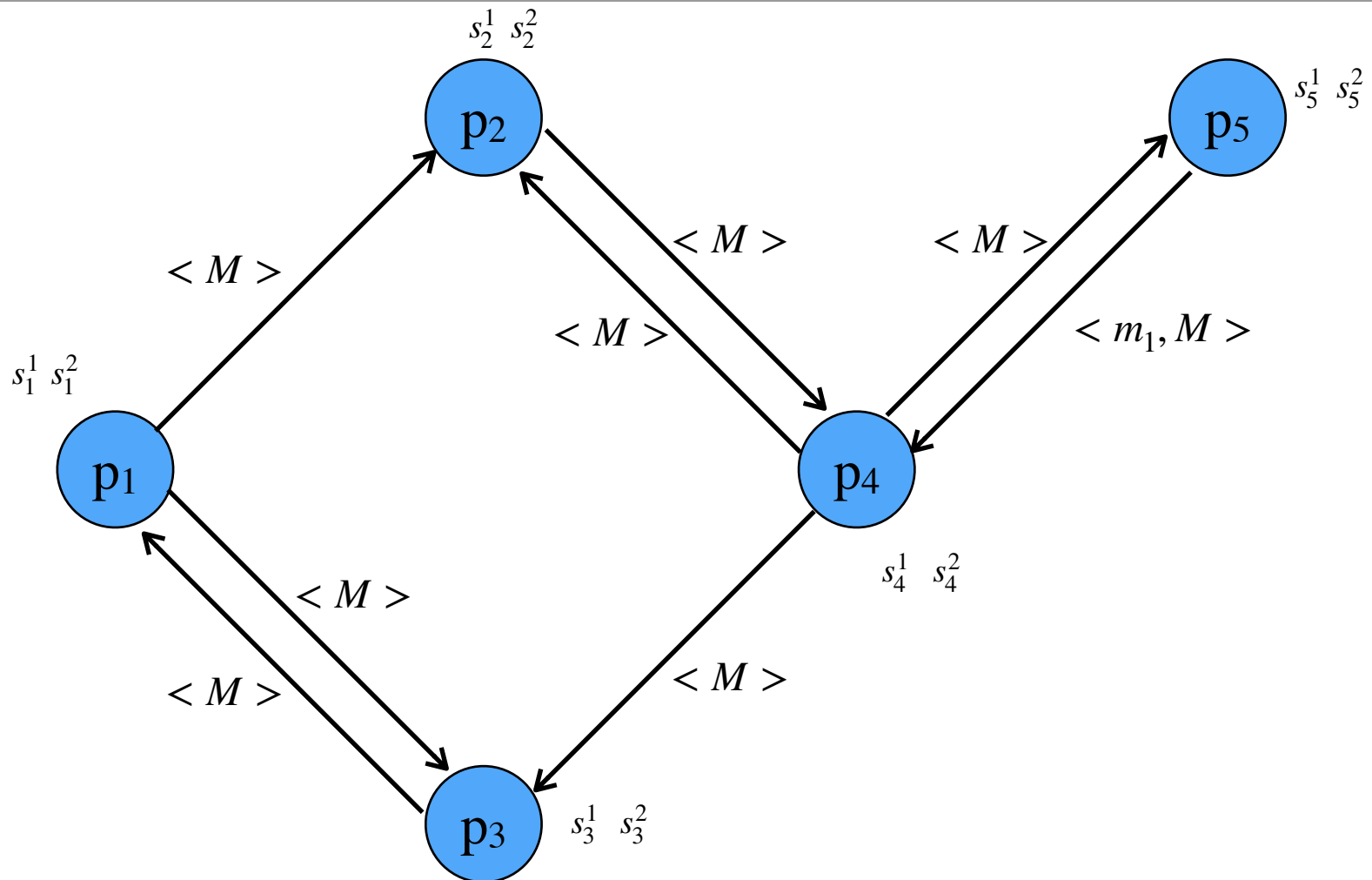
Larger example



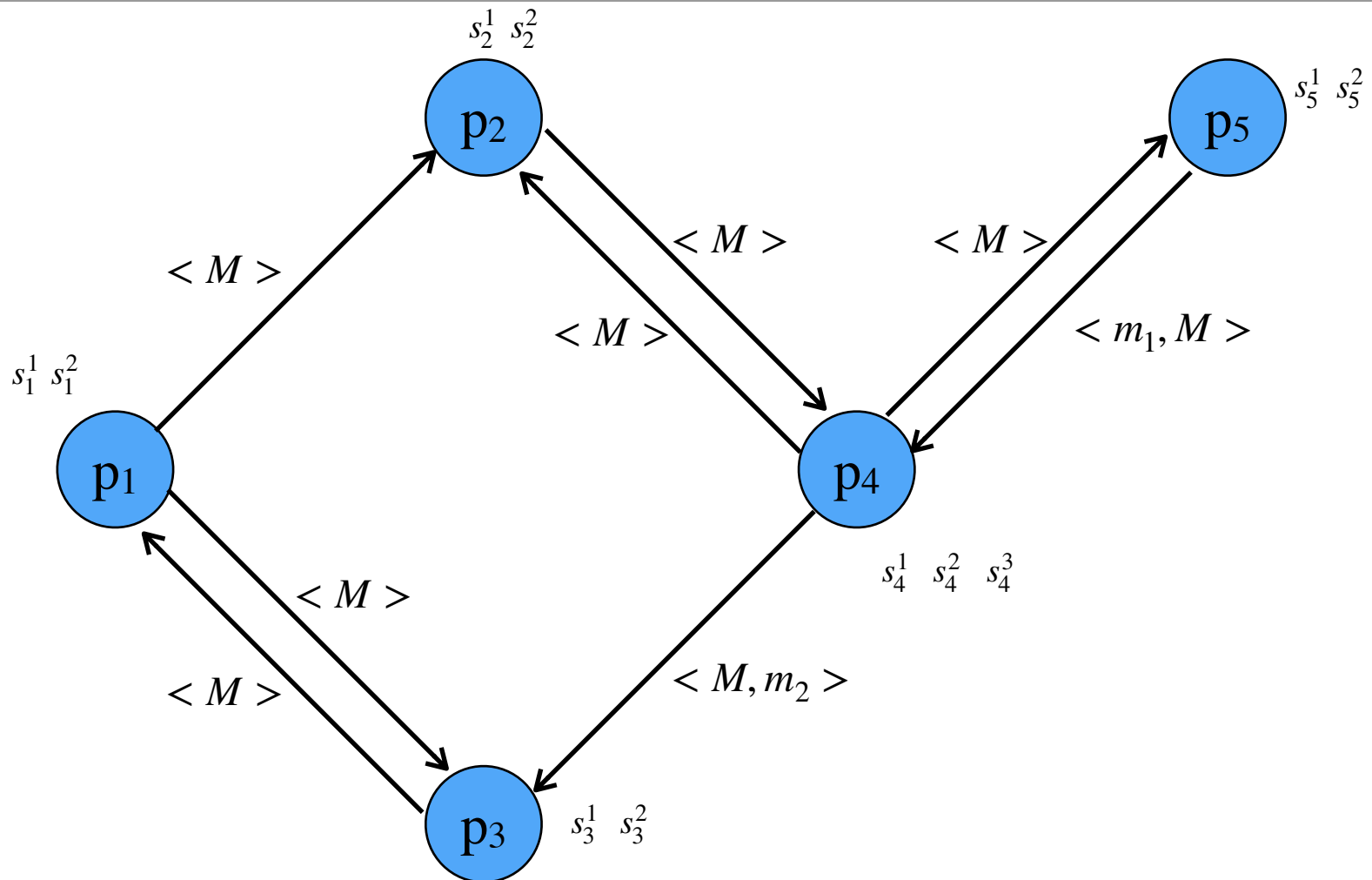
Larger example



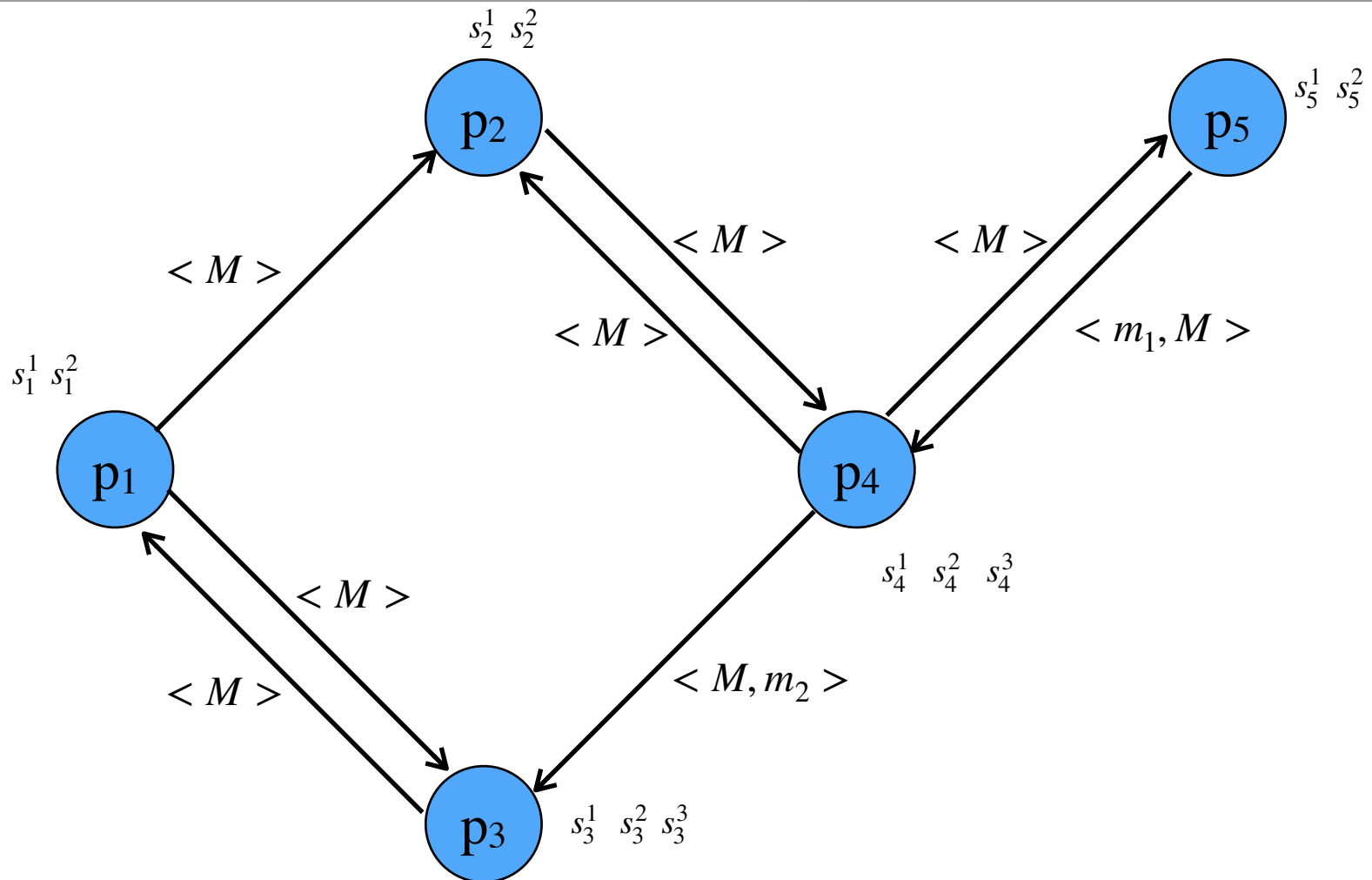
Larger example



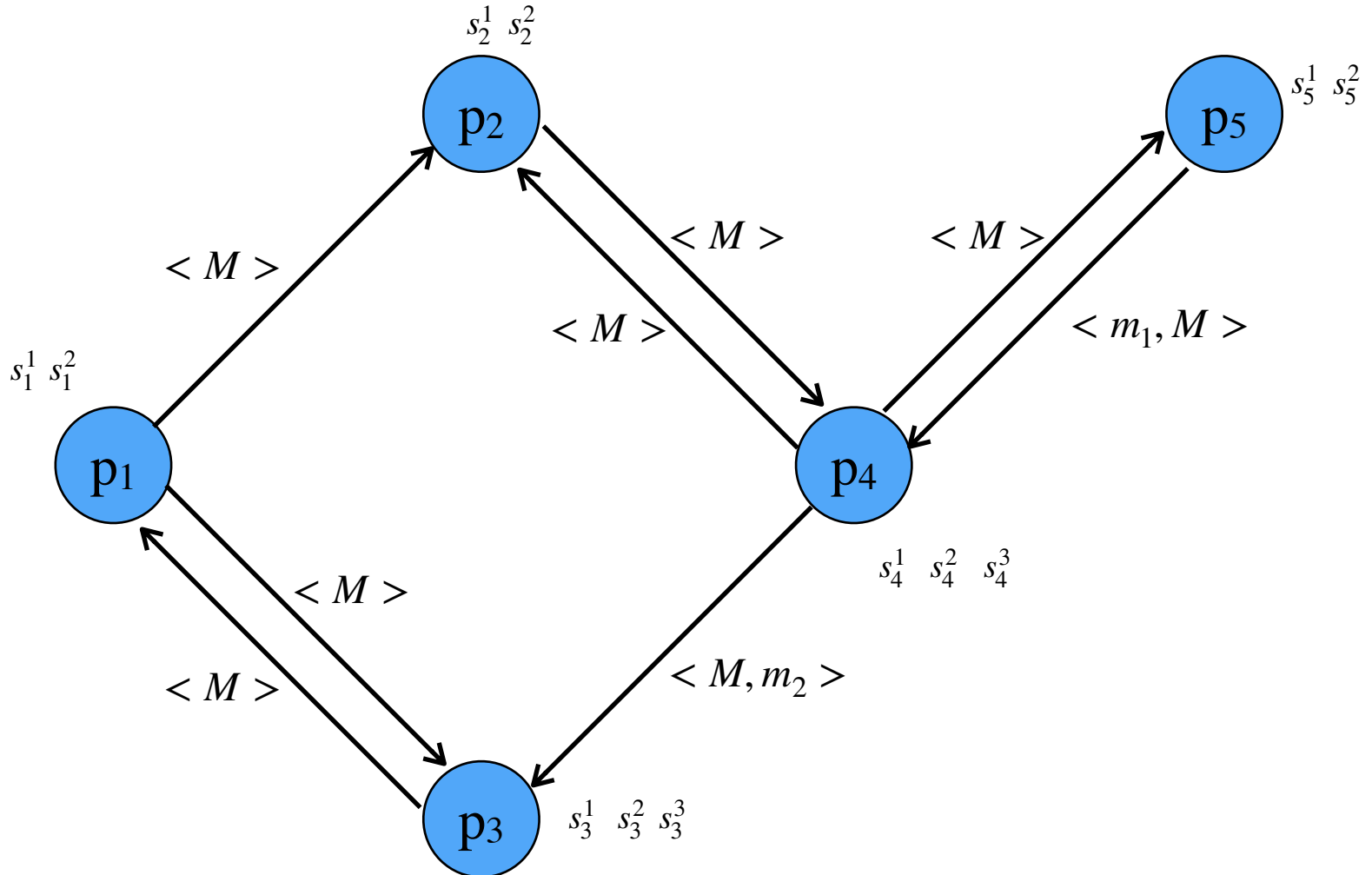
Larger example



Larger example



Larger example



Distributed recorded global state:

$$S^* = (s_1^1, s_1^1, s_3^1, s_4^1, s_5^2, c_{12} = \langle \rangle, c_{13} = \langle \rangle, c_{24} = \langle \rangle, c_{42} = \langle \rangle, c_{43} = \langle \rangle, c_{45} = \langle \rangle, c_{54} = \langle m_1 \rangle, c_{31} = \langle \rangle)$$

Correctness

Theorem: the distributed snapshot algorithm delivers a consistent global state.

Proof: Each p_i eventually records its local state: because of the graph's connectivity, all p_i s eventually receive a marker message.

We will now prove that the computed global state satisfies the following two conditions:

C1: Every message m_{ij} recorded as sent in the local state of p_i must be captured either in the state of channel c_{ij} it was sent over, or in the collected local state of the receiver p_j (conservation of messages).

C2: If an m_{ij} is not recorded as sent in the local state of p_i , then it must neither be present in the state of c_{ij} , nor in the collected local state of the receiver p_j (for every effect, its cause must be present).

Proof (continued)

Proof of C1: If a p_j receives a m_{ij} that precedes the marker $\langle M \rangle$ on channel c_{ij} , then if p_j has not taken a snapshot yet it includes m_{ij} in its recorded local state, otherwise it reports m_{ij} in the state of channel c_{ij} .

Proof of C2: If a m_{ij} is not included in the local state recorded by p_i , then it was sent after p_i had sent $\langle M \rangle$ over c_{ij} . Because channels are FIFO, p_j will receive $\langle M \rangle$ before m_{ij} , and thus it will report its local state before receiving m_{ij} and c_{ij} 's state as empty if this is the first marker it receives, or it will just stop recording c_{ij} again before receiving m_{ij} captured either in the state of channel c_{ij} it was sent over, or in the collected local state of the receiver p_j (conservation of messages).

Complexity

- Message complexity: $O(l)$, where l the number of links, plus the messages sent by the normal execution of the p_i s.
- Time complexity: $O(d)$ where d is the diameter of the network.